Slides adapted from Data 100, UC Berkeley

CS4048 — DATA SCIENCE

Lecture 01

Lecture 21

November 13, 2025

Fall 2025

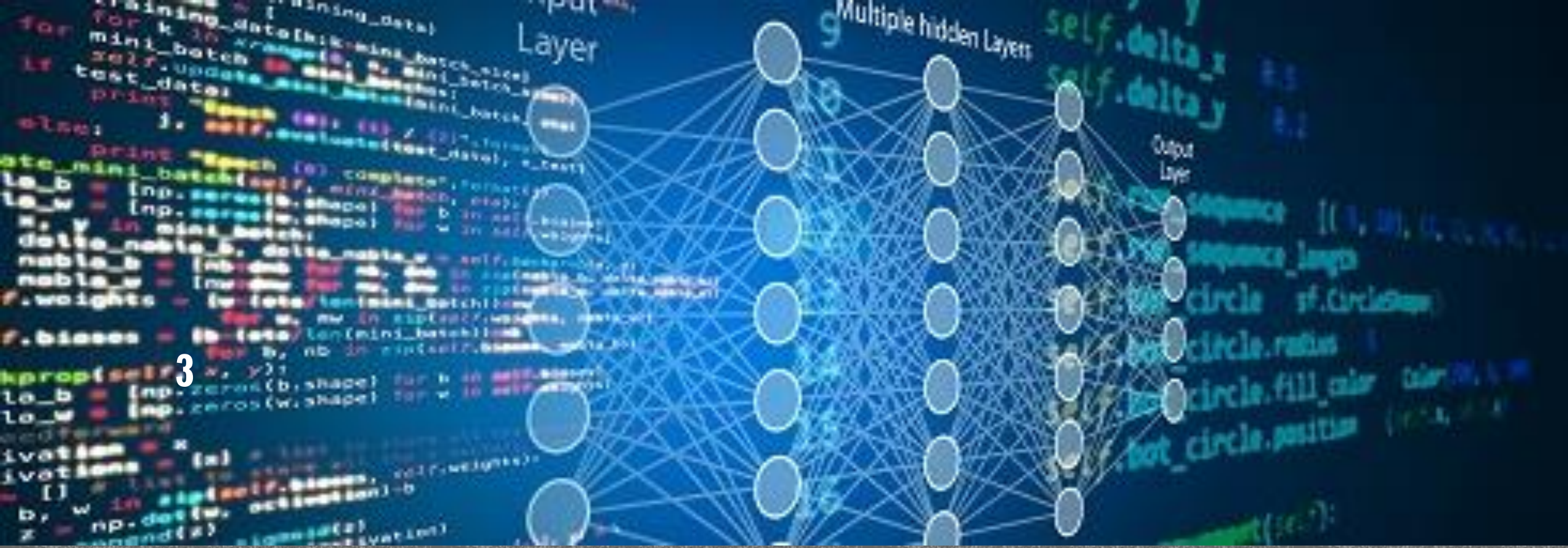
FAST – NUCES, CFD Campus

Dr. Rabia Maqsood

rabia.maqsood@nu.edu.pk

TODAY'S TOPICS

- Multiple Linear Regression
- Gradient descent



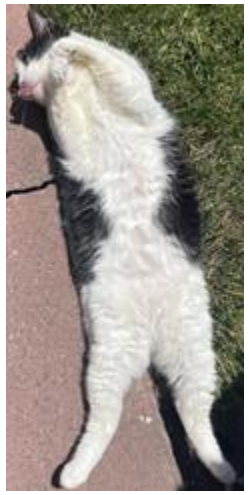
MULTIPLE LINEAR REGRESSION

EXTENDING SLR TO MORE THAN ONE INPUT

In SLR, we predict one output with one input.

In real prediction scenarios, we often want to account for more than one input.

Example: Predict cat's ages $\hat{y}$ as a linear model of 2 features: length x_1 **and** weight x_2 .



$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

intercept

parameter for length

parameter for weight

MULTIPLE LINEAR REGRESSION (I.E., ORDINARY LEAST SQUARES, OR OLS)

Define the **multiple linear regression** model:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_p x_p$$

Parameter vector $\theta = [\theta_0, \theta_1, \ldots, \theta_p]$

MULTIPLE LINEAR REGRESSION (I.E., ORDINARY LEAST SQUARES, OR OLS)

Define the **multiple linear regression** model:

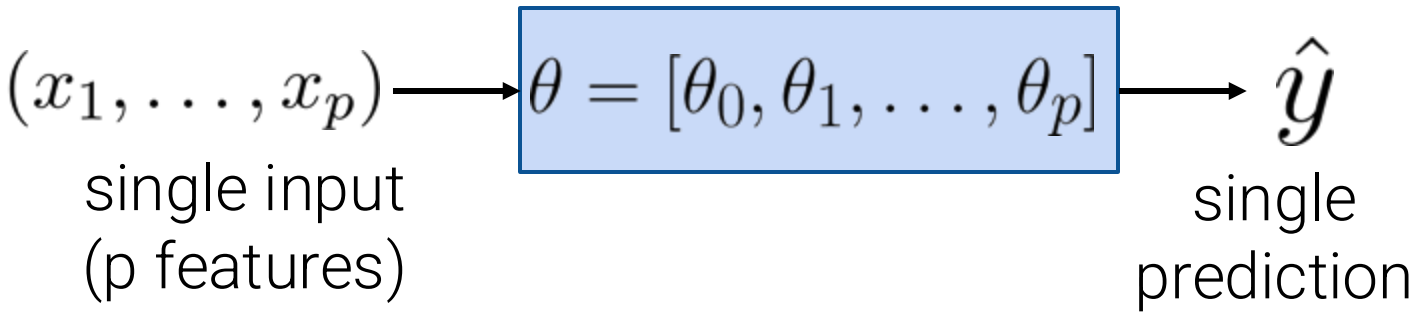
$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_p x_p$$

Parameter vector

$$\theta = [\theta_0, \theta_1, \ldots, \theta_p]$$

This is a **linear combination** of θ_j's , each scaled by x_j

In other words, $\hat{y}$ is linear in θ



MULTIPLE LINEAR REGRESSION

Define the **multiple linear regression** model:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_p x_p$$

Predicted value of y

This is a **linear combination** of θ_j 's, each scaled by x_j .

$$\vec{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_p \end{bmatrix} \rightarrow \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_p \end{bmatrix} \rightarrow \hat{y}$$

single **observation** (p features)

single prediction

MULTIPLE LINEAR REGRESSION

Define the **multiple linear regression** model:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_p x_p$$

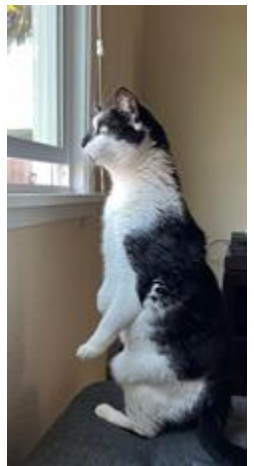
Predicted value of y

$$\vec{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_p \end{bmatrix} \rightarrow \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_p \end{bmatrix} \rightarrow \hat{y}$$

single **observation** (p features)

single prediction

Example: Predict cat's ages $\hat{y}$ as a linear model of 2 features: length x_1 **and** weight x_2 .



$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

intercept

parameter for length

parameter for weight

NBA 2018-2019 DATASET

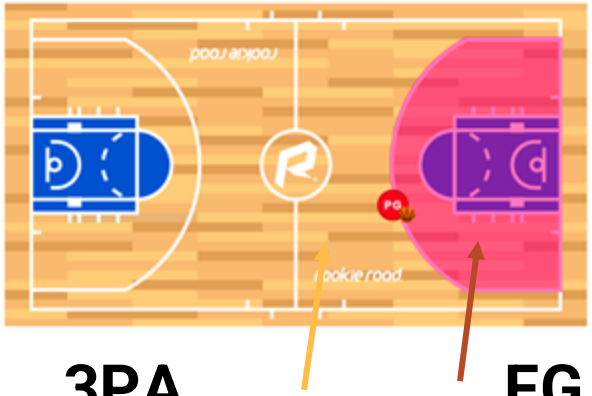
How many points does an athlete score per game?
PTS (average points/game)

To name a few factors:

- **FG:** average # 2 point field goals
- **AST:** average # of assists
- **3PA:** average # 3 point field goals attempted

	FG	AST	3PA	PTS
1	1.8	0.6	4.1	5.3
2	0.4	0.8	1.5	1.7
3	1.1	1.9	2.2	3.2
4	6.0	1.6	0.0	13.9
5	3.4	2.2	0.2	8.9
6	0.6	0.3	1.2	1.7

Rows correspond to individual players.



3PA **FG**

assist: a pass to a teammate that directly leads to a goal

MULTIPLE LINEAR REGRESSION MODEL

How many points does an athlete score per game?
PTS (average points/game)

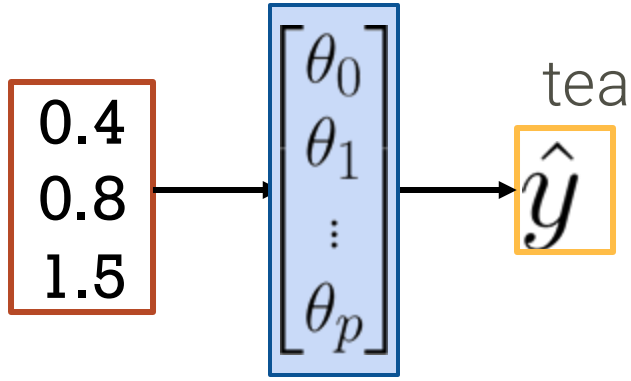
To name a few factors:

- **FG:** average # 2 point field goals
- **AST:** average # of assists
- **3PA:** average # 3 point field goals attempted

assist: a pass to a teammate that directly leads to a goal

	FG	AST	3PA	PTS
1	1.8	0.6	4.1	5.3
2	0.4	0.8	1.5	1.7
3	1.1	1.9	2.2	3.2
4	6.0	1.6	0.0	13.9
5	3.4	2.2	0.2	8.9
6	0.6	0.3	1.2	1.7

Rows correspond to individual players.



$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_p x_p$$
$$= \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$$

FG AST 3PA

TODAY'S GOAL: ORDINARY LEAST SQUARES

1. Choose a model	**Multiple Linear Regression**
2. Choose a loss function	L2 Loss **Mean Squared Error (MSE)**
3. Fit the model	Minimize average loss with ~~calculus~~ geometry
4. Evaluate model performance	Visualize, ~~Root MSE~~ Multiple R^2

In statistics, this model + loss is called **Ordinary Least Squares (OLS)**.

The solution to OLS are the minimizing loss for parameters $\hat{\theta}$, also called the **least squares estimate**.

TODAY'S GOAL: ORDINARY LEAST SQUARES

1. Choose a model

Multiple Linear Regression

For each of our n data points:

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_p x_p$$

$$\hat{\mathbb{Y}} = \mathbb{X}\theta$$

2. Choose a loss function

L2 Loss

Mean Squared Error (MSE)

3. Fit the model

Minimize average loss with ~~calculus~~ geometry

4. Evaluate model performance

Visualize,
~~Root MSE~~
Multiple R^2

FROM ONE FEATURE TO MANY FEATURES

Dataset for SLR

x	y
x_1	y_1
x_2	y_2
$\vdots$	$\vdots$
x_n	y_n

	FG	PTS
1	1.8	5.3
2	0.4	1.7
3	1.1	3.2
4	6.0	13.9
5	3.4	8.9
...	...	...

Dataset for Constant Model

y
y_1
y_2
$\vdots$
y_n

	PTS
1	5.3
2	1.7
3	3.2
4	13.9
5	8.9
...	...

Dataset for Multiple Linear Regression

$x_{:,1}$	$x_{:,2}$		$x_{:,p}$	y
x_{11}	x_{12}	$\cdots$	x_{1p}	y_1
x_{21}	x_{22}	$\cdots$	x_{2p}	y_2
$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
x_{n1}	x_{n2}	$\cdots$	x_{np}	y_n

	FG	AST	3PA	PTS
1	1.8	0.6	4.1	5.3
2	0.4	0.8	1.5	1.7
3	1.1	1.9	2.2	3.2
4	6.0	1.6	0.0	13.9
5	3.4	2.2	0.2	8.9
...	...	...	...	...

FROM ONE FEATURE TO MANY FEATURES

Dataset for Multiple Linear Regression

$x_{:1}$	$x_{:2}$		$x_{:p}$	y
x_{11}	x_{12}	$\cdots$	x_{1p}	y_1
x_{21}	x_{22}	$\cdots$	x_{2p}	y_2
$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
x_{n1}	x_{n2}	$\cdots$	x_{np}	y_n

Feature 2

$$\{x_{12}, x_{22}, \ldots x_{n2}\}$$

Observation i

$$\{x_{i1}, x_{i2}, \ldots x_{ip}, y_i\}$$

	FG	AST	3PA	PTS
1	1.8	0.6	4.1	5.3
2	0.4	0.8	1.5	1.7
3	1.1	1.9	2.2	3.2
4	6.0	1.6	0.0	13.9
5	3.4	2.2	0.2	8.9
...	...	...	...	...

Model

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_p x_p$$

$$\begin{cases} \hat{y}_1 = \theta_0 + \theta_1 x_{11} + \theta_2 x_{12} + \cdots + \theta_p x_{1p} \\ \hat{y}_2 = \theta_0 + \theta_1 x_{21} + \theta_2 x_{22} + \cdots + \theta_p x_{2p} \\ \vdots \\ \hat{y}_n = \theta_0 + \theta_1 x_{n1} + \theta_2 x_{n2} + \cdots + \theta_p x_{np} \end{cases}$$

[LINEAR ALGEBRA] VECTOR DOT PRODUCT

The **dot product (or inner product)** is a vector operation that

- Can only be carried out on two vectors of the **same length,**
- Sums up the products of the corresponding entries of the two vectors, and
- Returns a single number.

$$\vec{u} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \qquad \vec{v} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

$$\vec{u} \cdot \vec{v} = \vec{u}^\top \vec{v} = \vec{v}^\top \vec{u}$$
$$= 1 \cdot 1 + 2 \cdot 1 + 3 \cdot 1$$
$$= 6$$

"u dot v"

VECTOR NOTATION

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_p x_p$$

This part looks a little like a dot product…

We want to collect all the θ_i's into a single vector.

$$= \theta_0 + \begin{bmatrix} \theta_1 \\ \theta_2 \\ \vdots \\ \theta_p \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_p \end{bmatrix} = \theta_0 \cdot 1 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_p x_p$$

😖 What about this one???

VECTOR NOTATION

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_p x_p$$

$$= \theta_0 \cdot 1 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_p x_p$$

$$= \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_p \end{bmatrix} \cdot \begin{bmatrix} 1 \\ x_1 \\ x_2 \\ \vdots \\ x_p \end{bmatrix} = x^\top \theta$$

bias term, intercept term

We want to collect all the θ_i's into a single vector.

MATRIX NOTATION

We can now condense our earlier result:

$$\begin{cases} \hat{y}_1 = \theta_0 + \theta_1 x_{11} + \theta_2 x_{12} + \cdots + \theta_p x_{1p} \\ \hat{y}_2 = \theta_0 + \theta_1 x_{21} + \theta_2 x_{22} + \cdots + \theta_p x_{2p} \\ \vdots \\ \hat{y}_n = \theta_0 + \theta_1 x_{n1} + \theta_2 x_{n2} + \cdots + \theta_p x_{np} \end{cases}$$

$$\begin{cases} \hat{y}_1 = x_1^\top \theta \\ \hat{y}_2 = x_2^\top \theta \\ \vdots \\ \hat{y}_n = x_n^\top \theta \end{cases}$$

where $x_1^\top = [1 \quad x_{11} \quad x_{12} \quad \cdots \quad x_{1p}]$ is datapoint/observation 1

where $x_2^\top = [1 \quad x_{21} \quad x_{22} \quad \cdots \quad x_{2p}]$ is datapoint/observation 2

where $x_n^\top = [1 \quad x_{n1} \quad x_{n2} \quad \cdots \quad x_{np}]$ is datapoint/observation n

MATRIX NOTATION

	FG	AST	3PA	PTS
1	1.8	0.6	4.1	5.3
2	0.4	0.8	1.5	1.7
3	1.1	1.9	2.2	3.2
4	6.0	1.6	0.0	13.9
5	3.4	2.2	0.2	8.9
...	...	...	...	...

$$x_2 = \begin{bmatrix} 1 \\ 0.4 \\ 0.8 \\ 1.5 \end{bmatrix} \quad y_2 = 1.7 \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}$$

$$\begin{aligned} \hat{y}_2 &= x_2^\top \theta \\ &= \theta_0 + \theta_1 \cdot 0.4 + \theta_2 \cdot 0.8 + \theta_3 \cdot 1.5 \end{aligned}$$

Dimension check

$x_2 \in \mathbb{R}^4$ or $\mathbb{R}^{(p+1)}$

$\theta \in \mathbb{R}^4$ or $\mathbb{R}^{(p+1)}$

$y_2 \in \mathbb{R} \quad \hat{y}_2 \in \mathbb{R}$

also called scalars

$$\begin{cases} \hat{y}_1 = x_1^\top \theta \\ \hat{y}_2 = x_2^\top \theta \\ \vdots \\ \hat{y}_n = x_n^\top \theta \end{cases}$$

where $x_1^\top = [1 \quad x_{11} \quad x_{12} \quad \dots \quad x_{1p}]$ is datapoint/observation 1

where $x_2^\top = [1 \quad x_{21} \quad x_{22} \quad \dots \quad x_{2p}]$ is datapoint/observation 2

where $x_n^\top = [1 \quad x_{n1} \quad x_{n2} \quad \dots \quad x_{np}]$ is datapoint/observation n

MATRIX NOTATION

$$
\begin{aligned}
\hat{y}_1 &= \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1p} \end{bmatrix} \theta &&= x_1^T \theta \\
\hat{y}_2 &= \begin{bmatrix} 1 & x_{21} & x_{22} & \dots & x_{2p} \end{bmatrix} \theta &&= x_2^T \theta \\
&\vdots &&\vdots &&\vdots \\
\hat{y}_n &= \begin{bmatrix} 1 & x_{n1} & x_{n2} & \dots & x_{np} \end{bmatrix} \theta &&= x_n^T \theta
\end{aligned}
$$

n row vectors, each with dimension **(p+1)**

Expand each datapoint's transposed input

MATRIX NOTATION

$$\begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_n \end{bmatrix} = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1p} \\ 1 & x_{21} & x_{22} & \dots & x_{2p} \\ & & \vdots & & \\ 1 & x_{n1} & x_{n2} & \dots & x_{np} \end{bmatrix} \theta$$

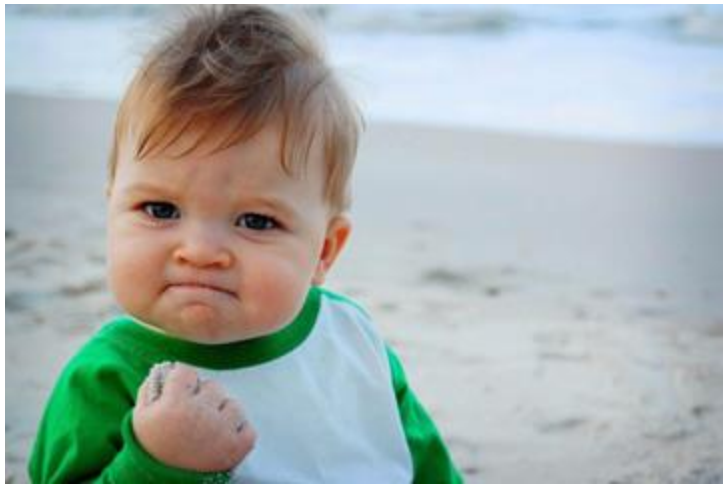
n row vectors, each with dimension **(p+1)**

Vectorize predictions and parameters to encapsulate all n equations into a single **matrix** equation.

MATRIX NOTATION

$$\begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_n \end{bmatrix} = \mathbb{X} \theta$$

Design matrix with dimensions n x (p + 1)



THE DESIGN MATRIX $\mathbb{X}$

We can use linear algebra to represent our predictions of all n data points at once.

One step in this process is to stack all of our input features together into a **design matrix**:

$$\mathbb{X} = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1p} \\ 1 & x_{21} & x_{22} & \dots & x_{2p} \\ 1 & x_{31} & x_{32} & \dots & x_{3p} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_{n1} & x_{n2} & \dots & x_{np} \end{bmatrix}$$

A **row** corresponds to one **observation**, e.g., all (p+1) features for datapoint 3

A **column** corresponds to a **feature**, e.g., feature 1 for all n data points

Special all-ones feature often called the **bias/intercept**

	Field Goals	Assists	3-Point Attempts	
Bias	FG	AST	3PA	PTS
1	1.8	0.6	4.1	5.3
1	0.4	0.8	1.5	1.7
1	1.1	1.9	2.2	3.2
1	6.0	1.6	0.0	13.9
1	3.4	2.2	0.2	8.9
...	...	...	...	...
1	4.0	0.8	0.0	11.5
1	3.1	0.9	0.0	7.8
1	3.6	1.1	0.0	8.9
1	3.4	0.8	0.0	8.5
1	3.8	1.5	0.0	9.4

Example design matrix 708 rows x (3+1) cols

THE MULTIPLE LINEAR REGRESSION MODEL USING MATRIX NOTATION

We can express our linear model on our entire dataset as follows:

$$\begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \hat{y}_3 \\ \vdots \\ \hat{y}_n \end{bmatrix} = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1p} \\ 1 & x_{21} & x_{22} & \dots & x_{2p} \\ 1 & x_{31} & x_{32} & \dots & x_{3p} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & x_{n1} & x_{n2} & \dots & x_{np} \end{bmatrix} \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_p \end{bmatrix}$$

$$\hat{\mathbb{Y}} = \mathbb{X}\theta$$

Prediction vector $\mathbb{R}^n$

Design matrix $\mathbb{R}^{n \times (p+1)}$

Parameter vector $\mathbb{R}^{(p+1)}$

Note that our **true output** is also a vector:

$$\mathbb{Y} \in \mathbb{R}^n$$

TODAY'S GOAL: ORDINARY LEAST SQUARES

1. Choose a model ✓	Multiple Linear Regression $\hat{\mathbb{Y}} = \mathbb{X}\theta$
2. Choose a loss function	L2 Loss Mean Squared Error (MSE) $R(\theta) = \frac{1}{n}\left(\|\mathbb{Y} - \hat{\mathbb{Y}}\|_2\right)^2$
3. Fit the model	Minimize average loss with ~~calculus~~ geometry
4. Evaluate model performance	Visualize, ~~Root MSE~~ Multiple R^2

[LINEAR ALGEBRA] VECTOR NORMS AND THE L2 VECTOR NORM

The **norm** of a vector is some measure of that vector's **size/length**.

- The two norms we need to know for this course are the L_1 and L_2 norms (sound familiar?).
- Today, we focus on L_2 norm. We'll define the L_1 norm another day.

For the n-dimensional vector $\vec{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$, the **L2 vector norm** is

$$||\vec{x}||_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2} = \sqrt{\sum_{i=1}^{n} (x_i^2)}$$

MEAN SQUARED ERROR WITH L2 NORMS

We can rewrite mean squared error as a squared L2 norm:

$$R(\theta) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$= \frac{1}{n}\left(\|\mathbb{Y} - \hat{\mathbb{Y}}\|_2\right)^2$$

$$||\vec{x}||_2 = \sqrt{\sum_{i=1}^{n}(x_i^2)}$$

With our linear model $\hat{\mathbb{Y}} = \mathbb{X}\theta$:

$$R(\theta) = \frac{1}{n}\left(\|\mathbb{Y} - \hat{\mathbb{Y}}\|_2\right)^2$$

LEAST SQUARES ESTIMATE

1. Choose a model ✓

Multiple Linear Regression

$$\hat{\mathbb{Y}} = \mathbb{X}\theta$$

2. Choose a loss function ✓

L2 Loss

Mean Squared Error (MSE)

$$R(\theta) = \frac{1}{n}\left(\|\mathbb{Y} - \hat{\mathbb{Y}}\|_2\right)^2$$

3. Fit the model ✓

Minimize average loss with ~~calculus~~ ~~geometry~~

$$\hat{\theta} = \left(\mathbb{X}^\mathsf{T}\mathbb{X}\right)^{-1}\mathbb{X}^\mathsf{T}\mathbb{Y}$$

4. Evaluate model performance

Visualize,
~~Root MSE~~
Multiple R^2

LEAST SQUARES ESTIMATE

1. Choose a model ✓

Multiple Linear Regression

$$\hat{\mathbb{Y}} = \mathbb{X}\theta$$

2. Choose a loss function ✓

L2 Loss

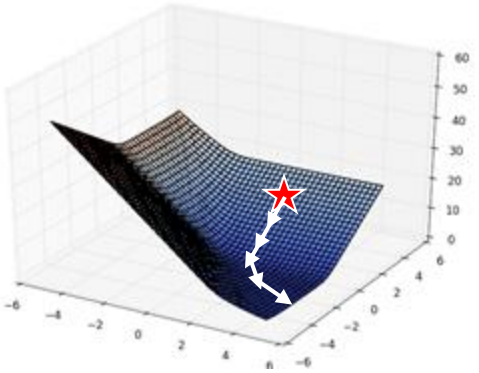
Mean Squared Error (MSE)

$$R(\theta) = \frac{1}{n}\left(\|\mathbb{Y} - \hat{\mathbb{Y}}\|_2\right)^2$$

3. Fit the model

Minimize average loss with **calculus, geometry, algorithms!**

We'll discuss shortly



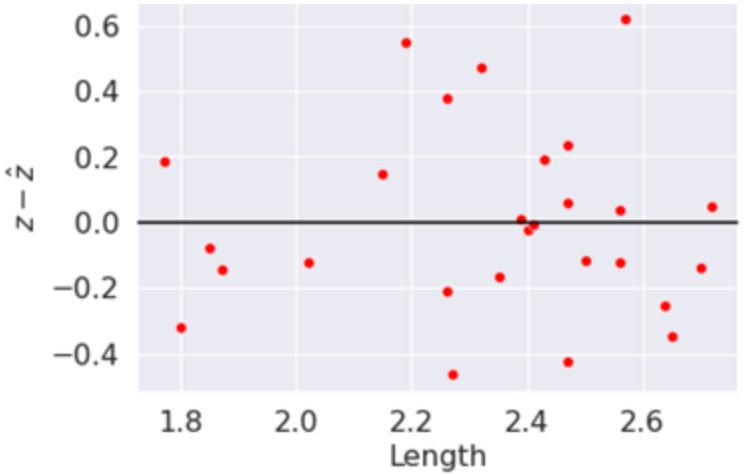
4. Evaluate model performance

Visualize,
~~Root MSE~~
Multiple R^2

[VISUALIZATION] RESIDUAL PLOTS

Simple linear regression

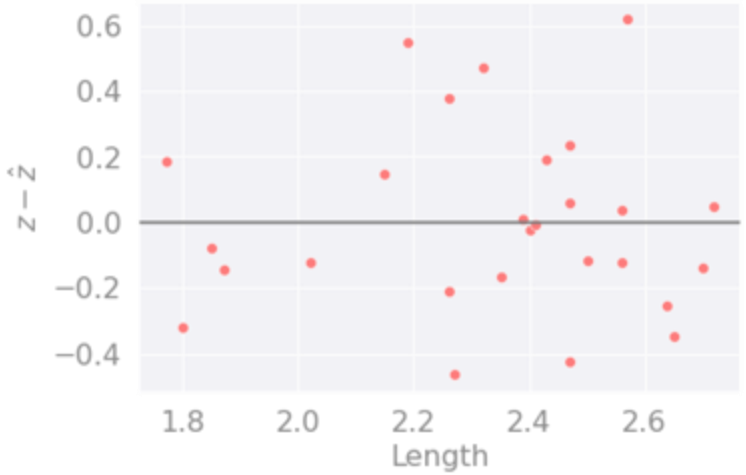
Plot residuals vs
the single feature X.



[VISUALIZATION] RESIDUAL PLOTS

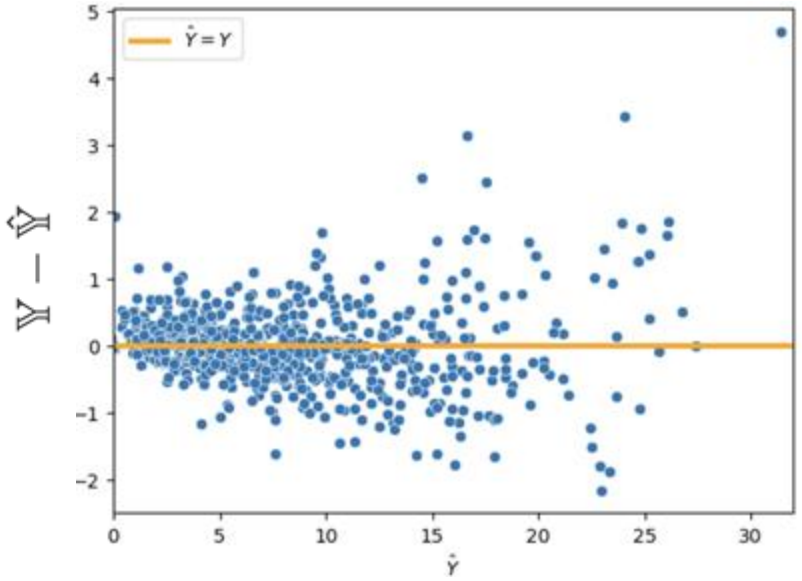
Simple linear regression

Plot residuals vs
the single feature X.



Multiple linear regression

Plot residuals vs
fitted (predicted) values $\hat{y}$.



Same interpretation as before:

- A good residual plot shows no pattern, and has a similar vertical spread across the plot (homoscedasticity).
- If **heteroscedasticity** (e.g., spreading out/in when moving across plot), the accuracy of the predictions is not reliable.

[METRICS] MULTIPLE R^2

Simple linear regression

Error
RMSE

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Linearity
Correlation coefficient, *r*

$$r = \frac{1}{n}\sum_{i=1}^{n}\left(\frac{x_i - \bar{x}}{\sigma_x}\right)\left(\frac{y_i - \bar{y}}{\sigma_y}\right)$$

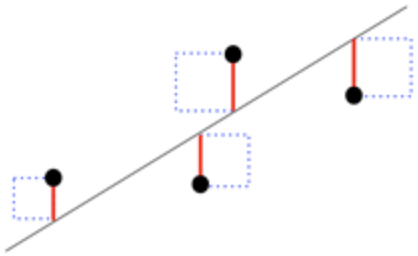
Multiple linear regression

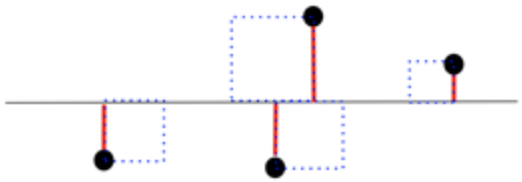
Error
RMSE

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Linearity
Multiple R^2, also called the **coefficient of determination**

$$R^2 = \frac{\text{variance of fitted values}}{\text{variance of } y} = \frac{\sigma_{\hat{y}}^2}{\sigma_y^2}$$





[METRICS] MULTIPLE R^2

We define the **multiple R^2** value as the **proportion of variance** or our **fitted values** (predictions) $\hat{y}$ to our true values y.

$$R^2 = \frac{\text{variance of fitted values}}{\text{variance of } y} = \frac{\sigma^2_{\hat{y}}}{\sigma^2_y}$$

Also called the **coefficient of determination**.

R^2 ranges from 0 to 1 and is effectively
"the proportion of variance that the **model explains**."

For OLS with an intercept term (e.g. $\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_p x_p$),
$R^2 = [r(y, \hat{y})]^2$ is equal to the square of correlation between y , $\hat{y}$.

- For SLR, $R^2 = r^2$, the correlation between x,y .
- The proof of these last two properties is beyond this course.

[METRICS] MULTIPLE R^2

As we add more features, our fitted values tend to become closer and closer to our actual y values. Thus, R^2 increases.

- The SLR model (AST only) explains 45.7% of the variance in the true y.

.

- The AST & 3PA model explains 60.9%.

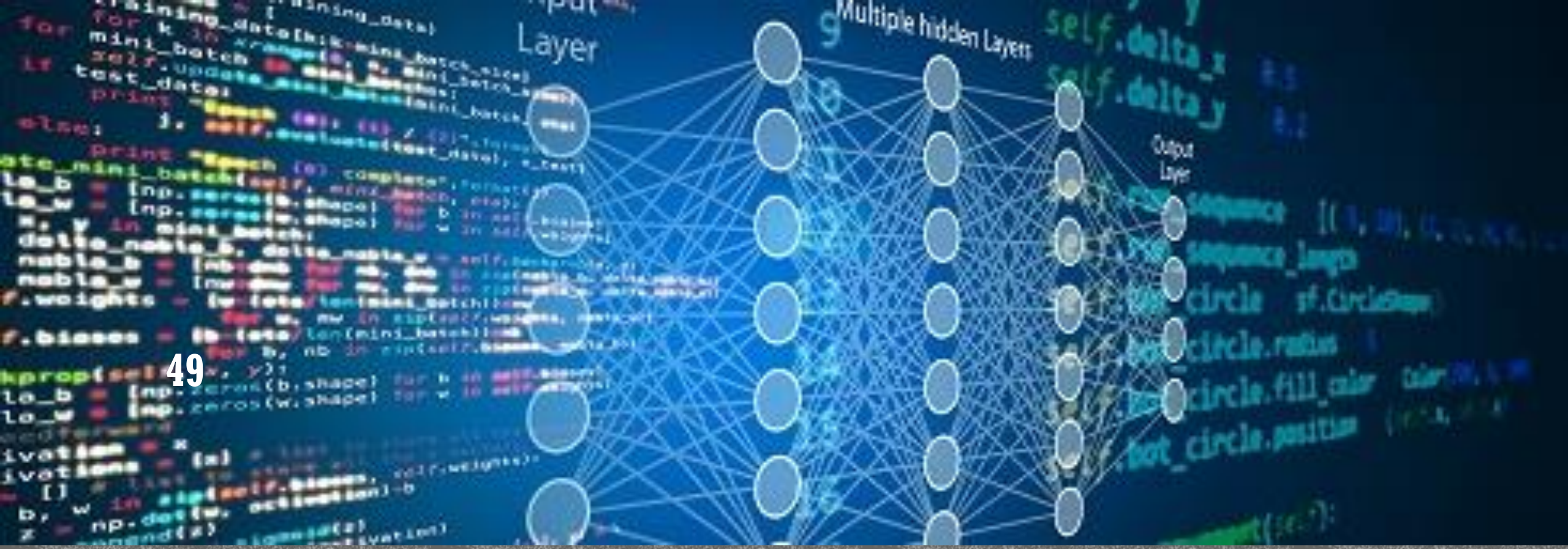
Adding more features doesn't always mean our model is better, though! We will see why later in the course.

$$\text{predicted PTS} = 3.98 + 2.4 \cdot \text{AST}$$

$R^2 = 0.457$

$$\text{predicted PTS} = 2.163 + 1.64 \cdot \text{AST} + 1.26 \cdot \text{3PA}$$

$R^2 = 0.609$



GRADIENT DESCENT

LEAST SQUARES ESTIMATE

1. Choose a model ✓

Multiple Linear Regression

$$\hat{\mathbb{Y}} = \mathbb{X}\theta$$

2. Choose a loss function ✓

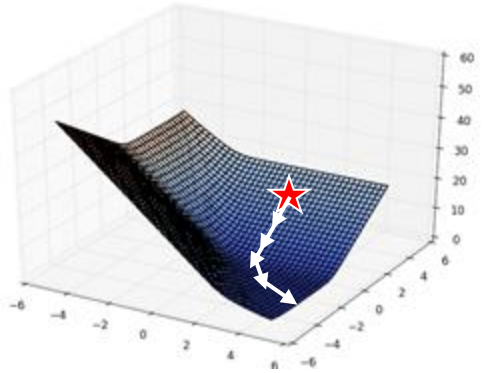
L2 Loss

Mean Squared Error (MSE)

$$R(\theta) = \frac{1}{n}\left(\|\mathbb{Y} - \hat{\mathbb{Y}}\|_2\right)^2$$

3. Fit the model

Minimize average loss with **calculus, geometry, algorithms!**

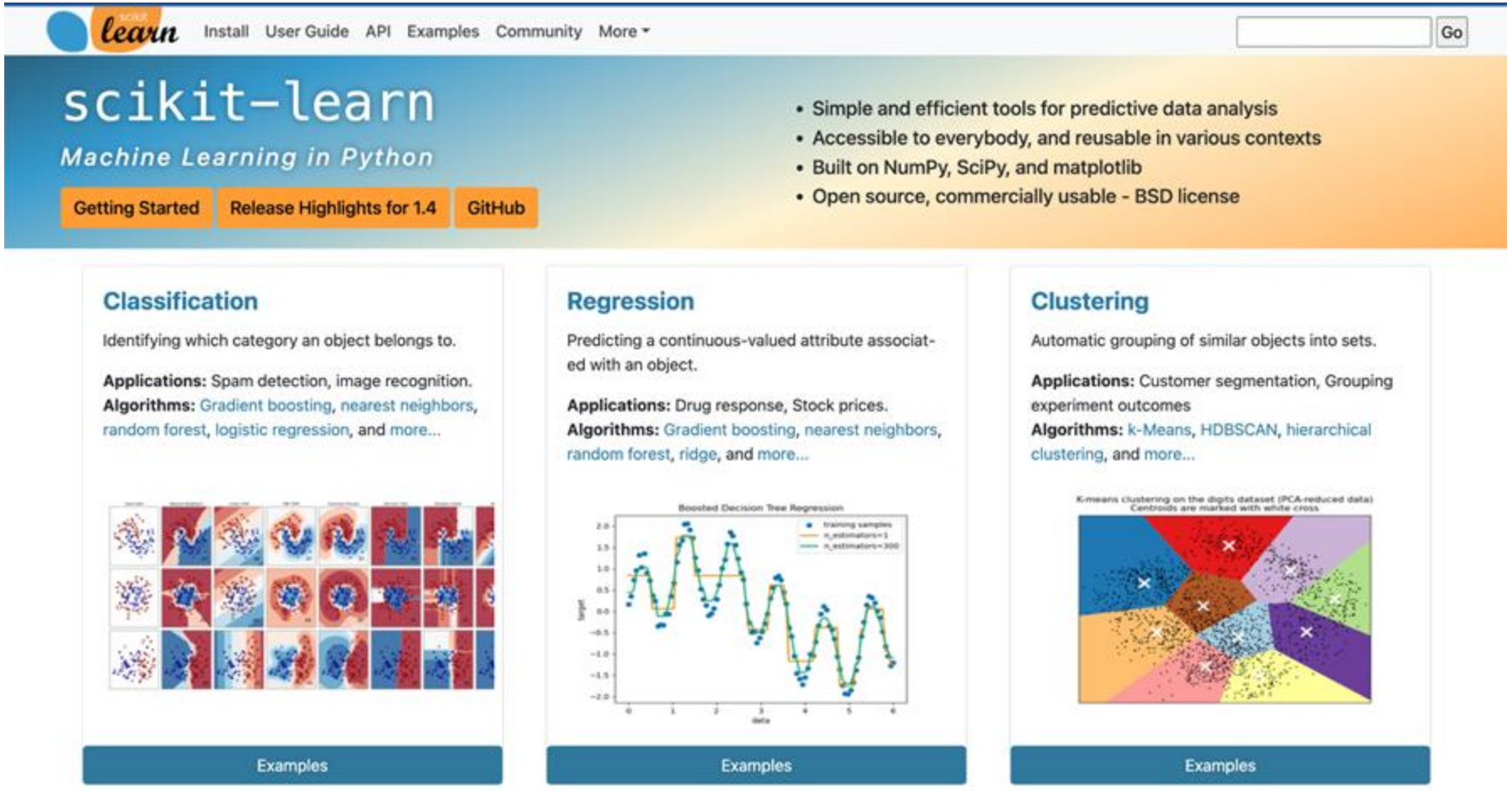


4. Evaluate model performance

Visualize,
~~Root MSE~~
Multiple R^2

MACHINE LEARNING (ML) FRAMEWORKS

In research and industry, it is common to use **libraries for creating and training models.**
We will use `scikit-learn`, commonly called `sklearn`



Other ML libraries:

- XGboost
- Pytorch
- TensorFlow
- 🤗 Hugging Face

`scikit-learn` has many great tutorials and is the standard for simple ML tasks.

THE SKLEARN WORKFLOW

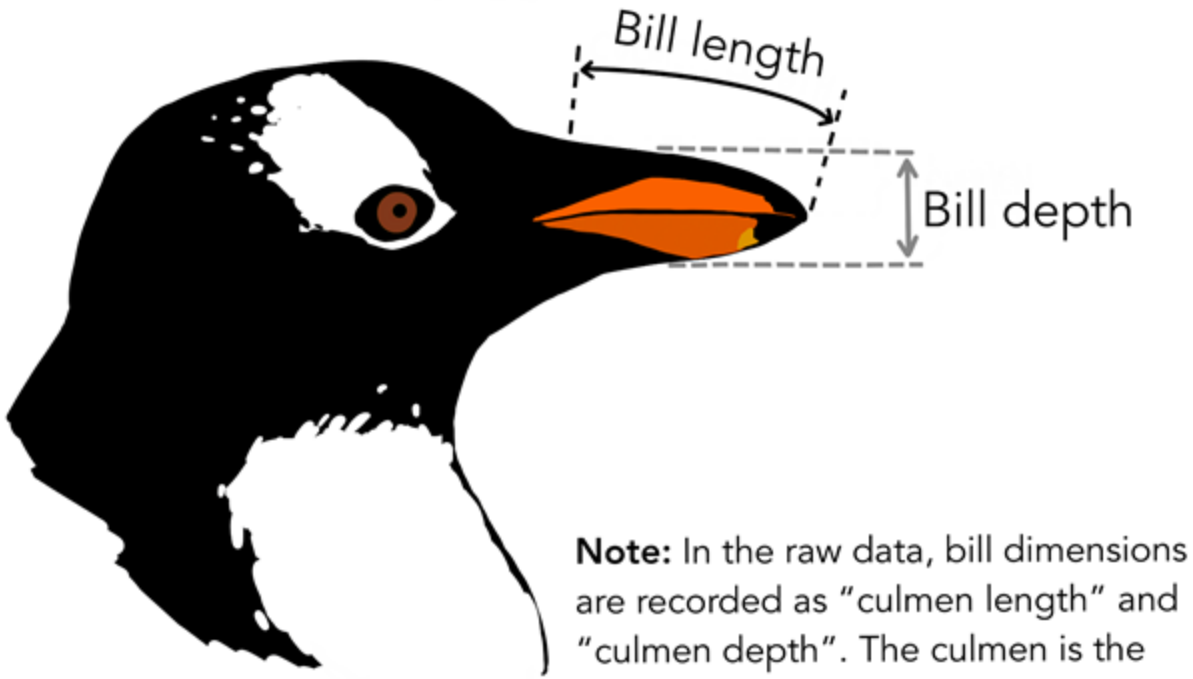
At a high level, there are three steps to creating an `sklearn` model:

1	**Initialize** a new model instance *Make a "copy" of the model template*	`my_model = lm.LinearRegression()`
2	**Fit** the model to the training data *Save the optimal model parameters*	`my_model.fit(X, y)`
3	Use fitted model to make **predictions**	`my_model.predict(X_new)`

To extract the fitted model's **parameters**: `my_model.coef_` and `my_model.intercept_`

FITTING AN OLS MODEL WITH SKLEARN

Do we obtain the same fitted OLS parameters provided by the normal equation using `sklearn`?



Note: In the raw data, bill dimensions are recorded as "culmen length" and "culmen depth". The culmen is the dorsal ridge atop the bill.

- lec13.ipynb

SCIKIT-LEARN DEMO SUMMARY

1. Create an `sklearn` object.

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model
```

```
▾ LinearRegression
LinearRegression()
```
Not fitted

2. `fit` the object to data.

X is a **DataFrame** of (n_samples, n_features), hence the double brackets.

y is a **Series** of (n_samples,), observations to predict.

```
model.fit(
    X=df[["flipper_length_mm", "body_mass_g"]],
    y=df["bill_depth_mm"])
```

```
▾ LinearRegression
LinearRegression()
```
Fitted

3. Analyze fit or call `predict`.

```
model.predict(df[["flipper_length_mm", "body_mass_g"]])
```

```
model.intercept_      # why is this a scalar?
```

```
11.00299527744707
```

```
model.coef_           # why is this an array?
```

```
array([0.00982849, 0.0014775 ])
```

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

WHERE WE'RE GOING

Big assumptions with the calculus-based and geometric techniques for minimizing loss:

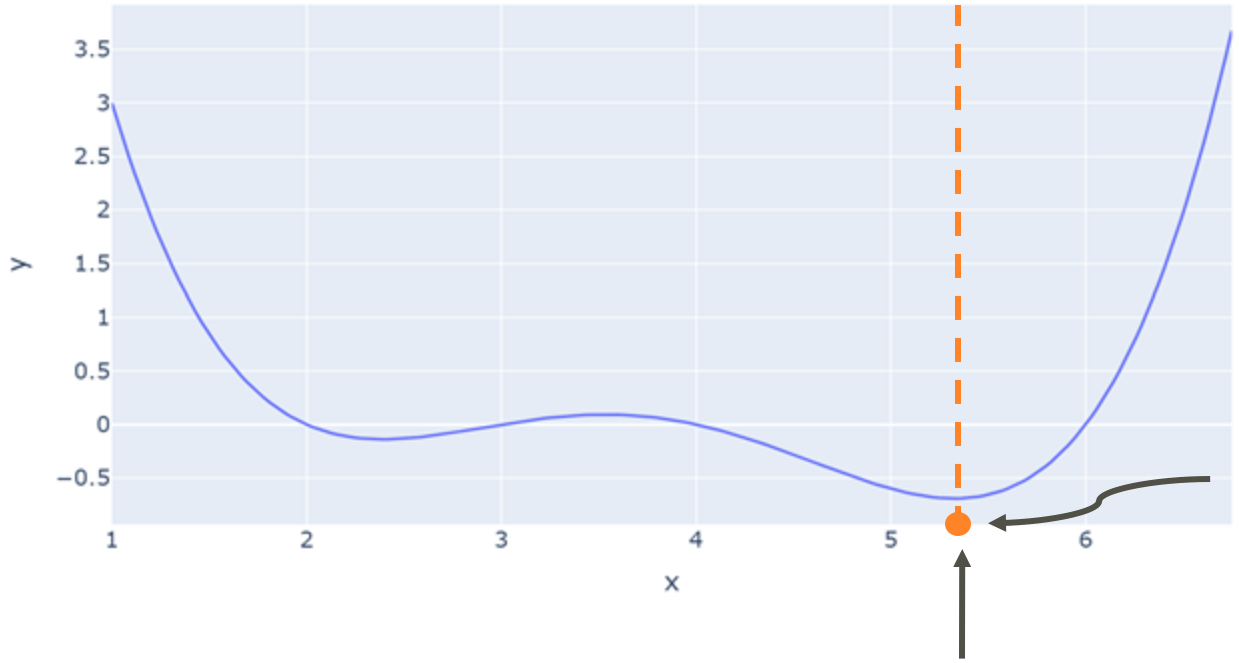
- **Calculus:** We can't always algebraically solve for the zero points of the derivative.
- **Geometric:** We don't always use a linear model with MSE loss.

To design more complex models with different loss functions, we need a new optimization technique: **gradient descent**

Big Idea: Use an **iterative algorithm** to numerically compute the **minimum of the loss.**

MINIMIZING AN ARBITRARY 1D FUNCTION

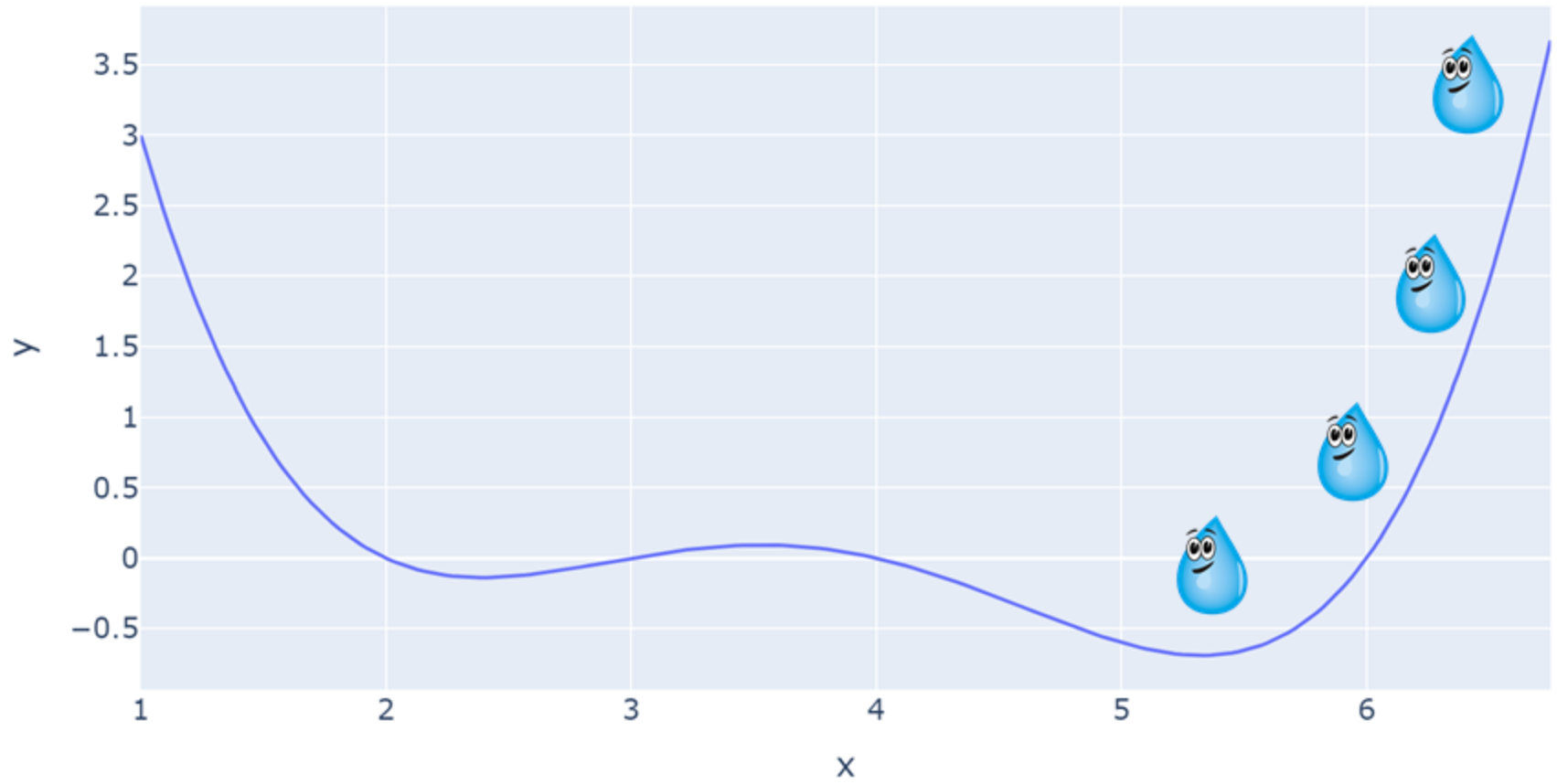
How do we find the value of x that minimizes this function?



- lec13.ipynb

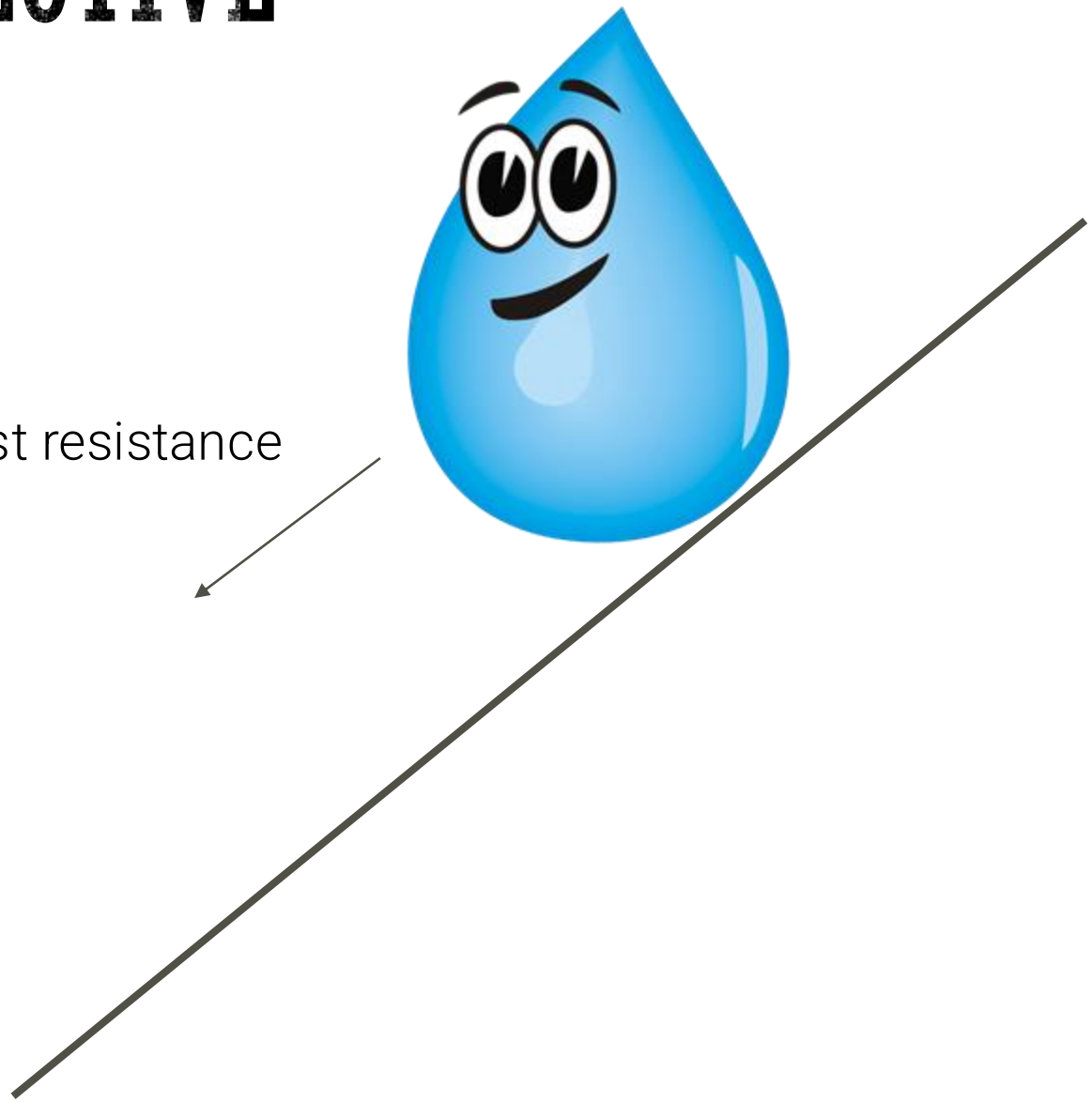
FINDING THE MINIMUM

How does a droplet of water "know" how to reach the bottom?



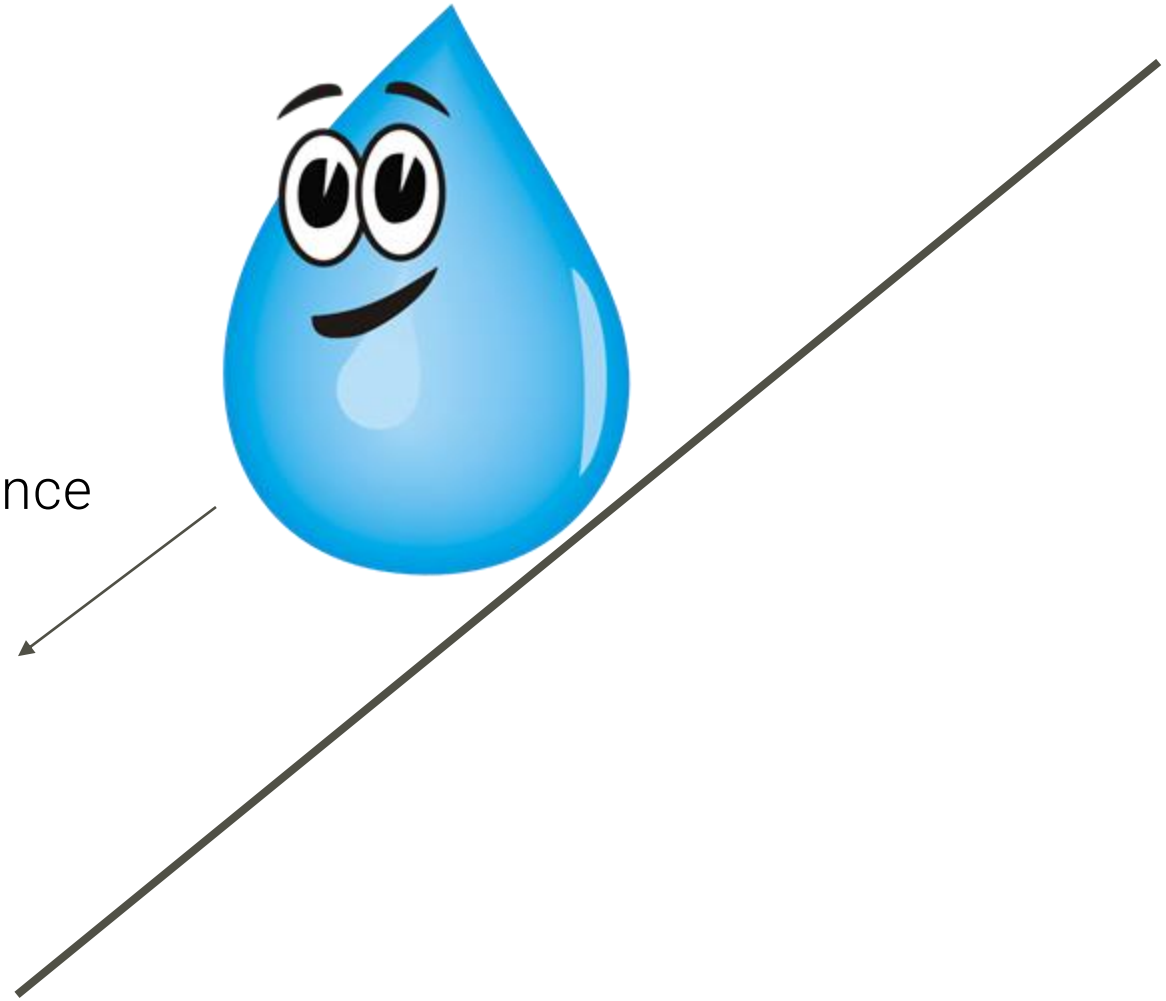
THE DROPLET'S PERSPECTIVE

Path of least resistance

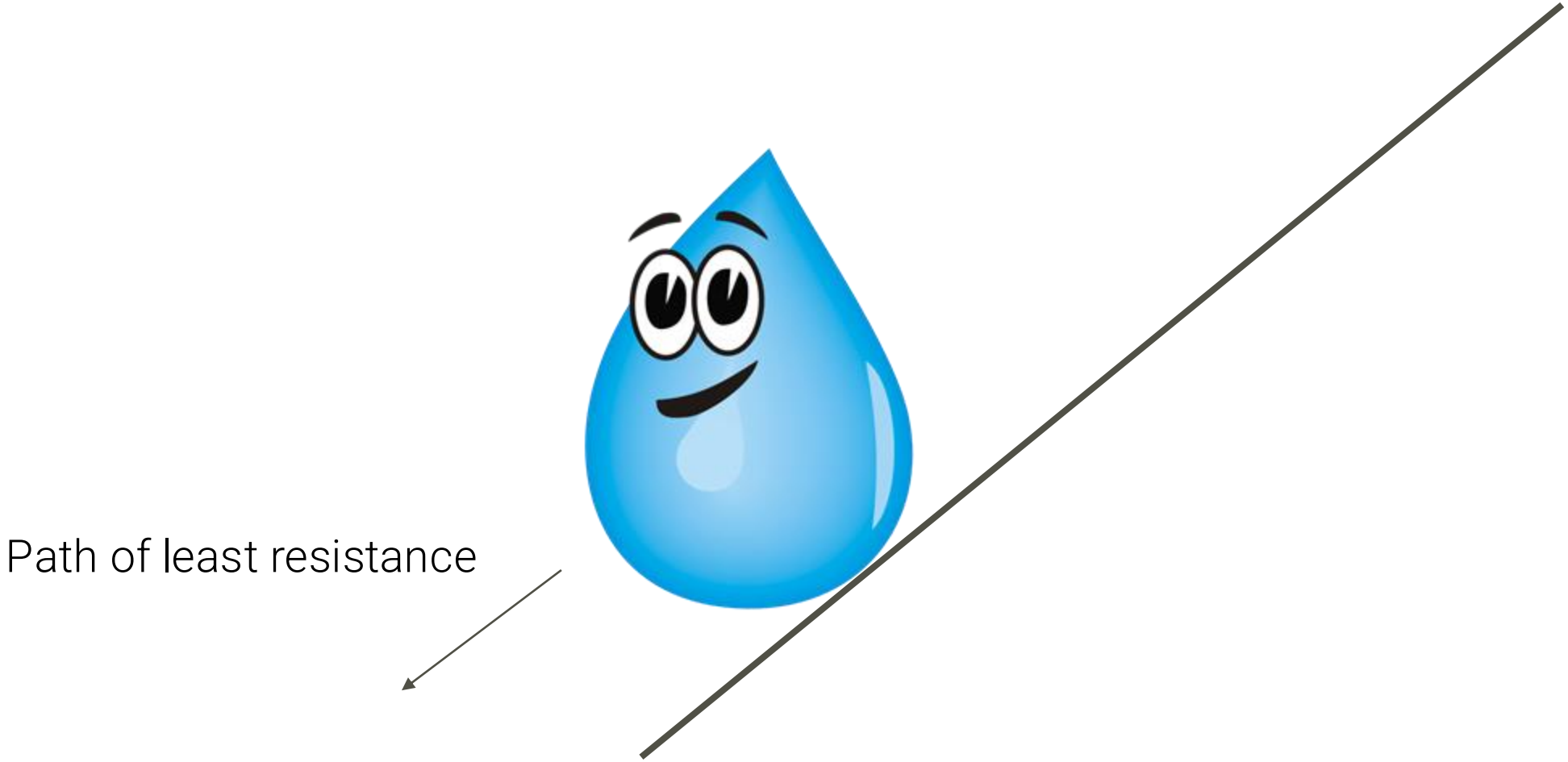


THE DROPLET'S PERSPECTIVE

Path of least resistance



THE DROPLET'S PERSPECTIVE



THE DROPLET'S PERSPECTIVE

THE DROPLET'S PERSPECTIVE

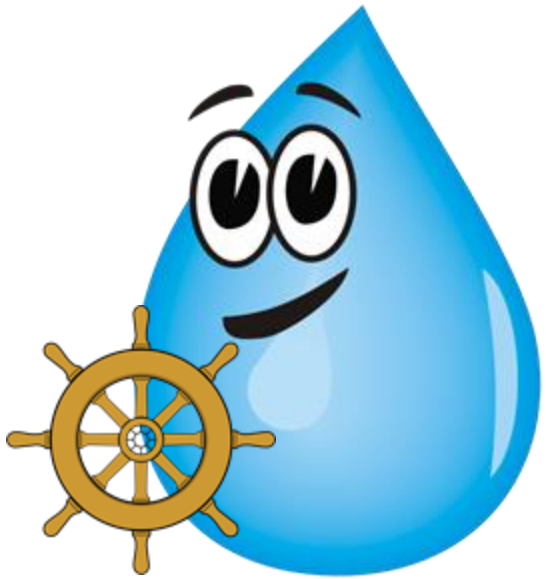
Path of least resistance: **Stay put!**

DRIVING INSTRUCTIONS FOR THE DROPLET

1. **Face** the path of least resistance.

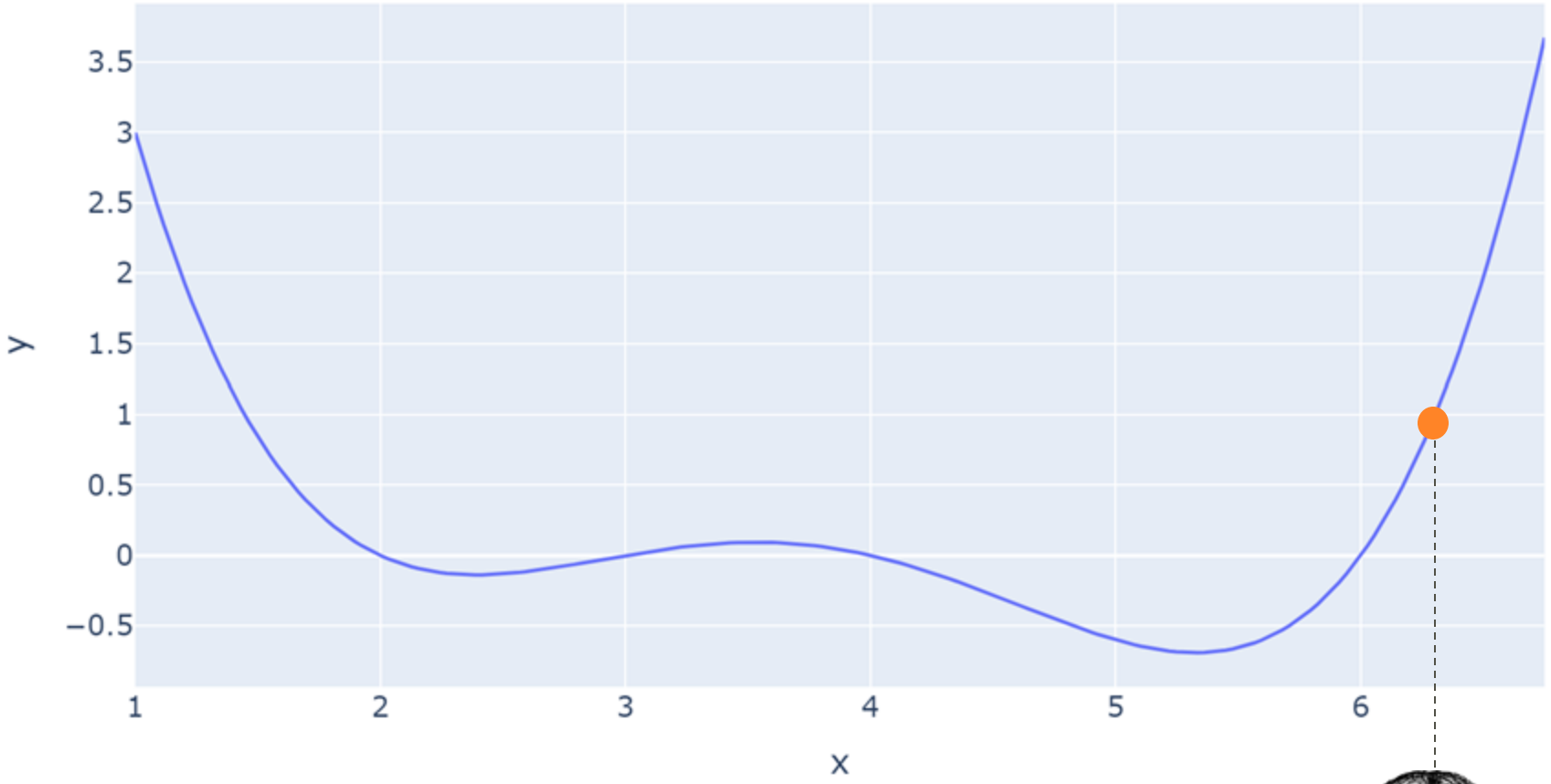
1. **Move** in that direction a little bit.

1. **Iterate** until the path of least resistance is to stay put.



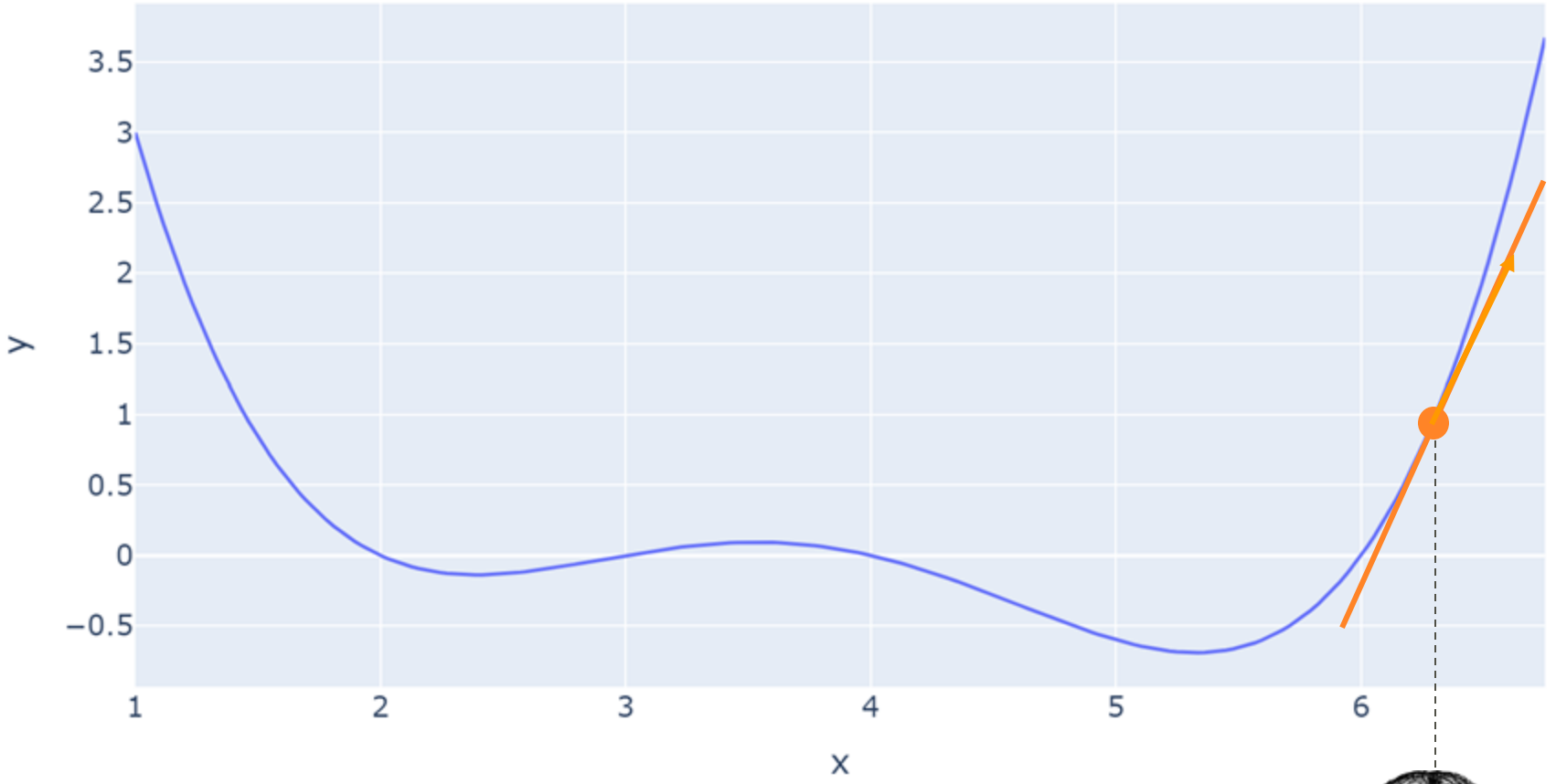
FINDING THE MINIMUM, REDUX

We could start with a random guess.



FINDING THE MINIMUM, REDUX

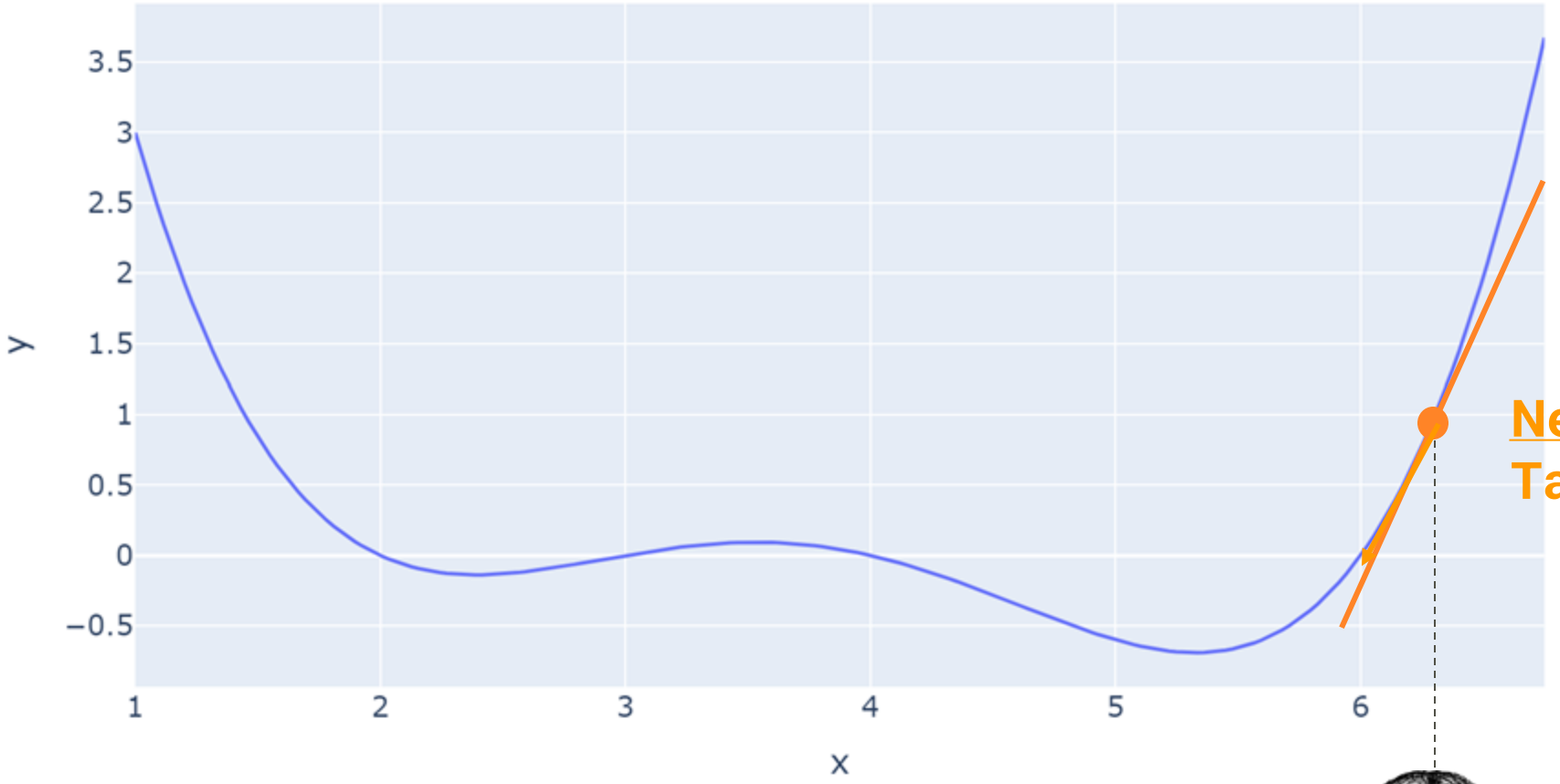
We calculate the tangent slope at that random point by taking a **derivative**.



Tangent Slope

FINDING THE MINIMUM, REDUX

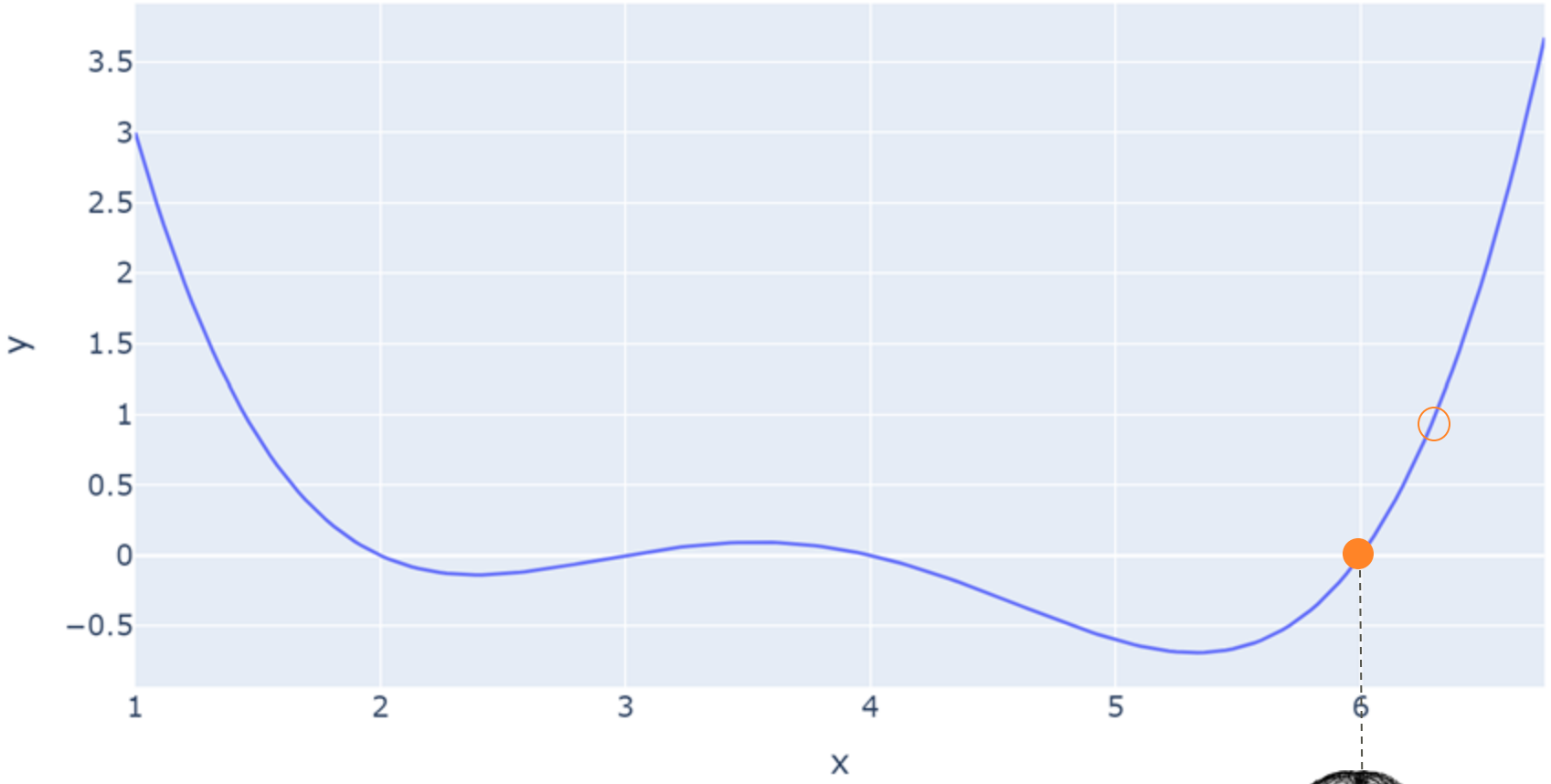
Downhill is in the **opposite** direction of the tangent slope.



Negative
Tangent Slope

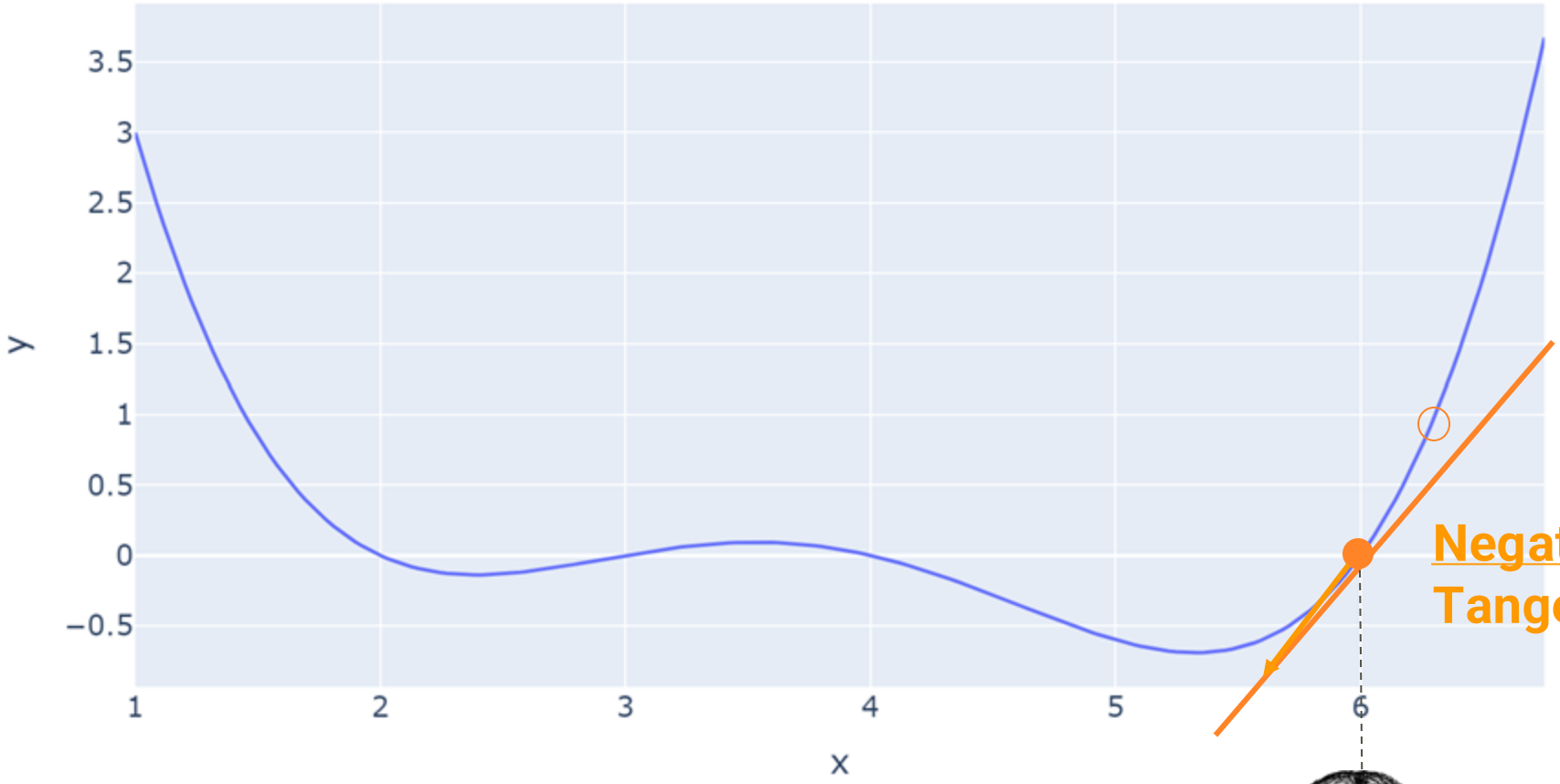
FINDING THE MINIMUM, REDUX

We take a step **left** to move our point **downhill.** (How big of a step? We choose!)



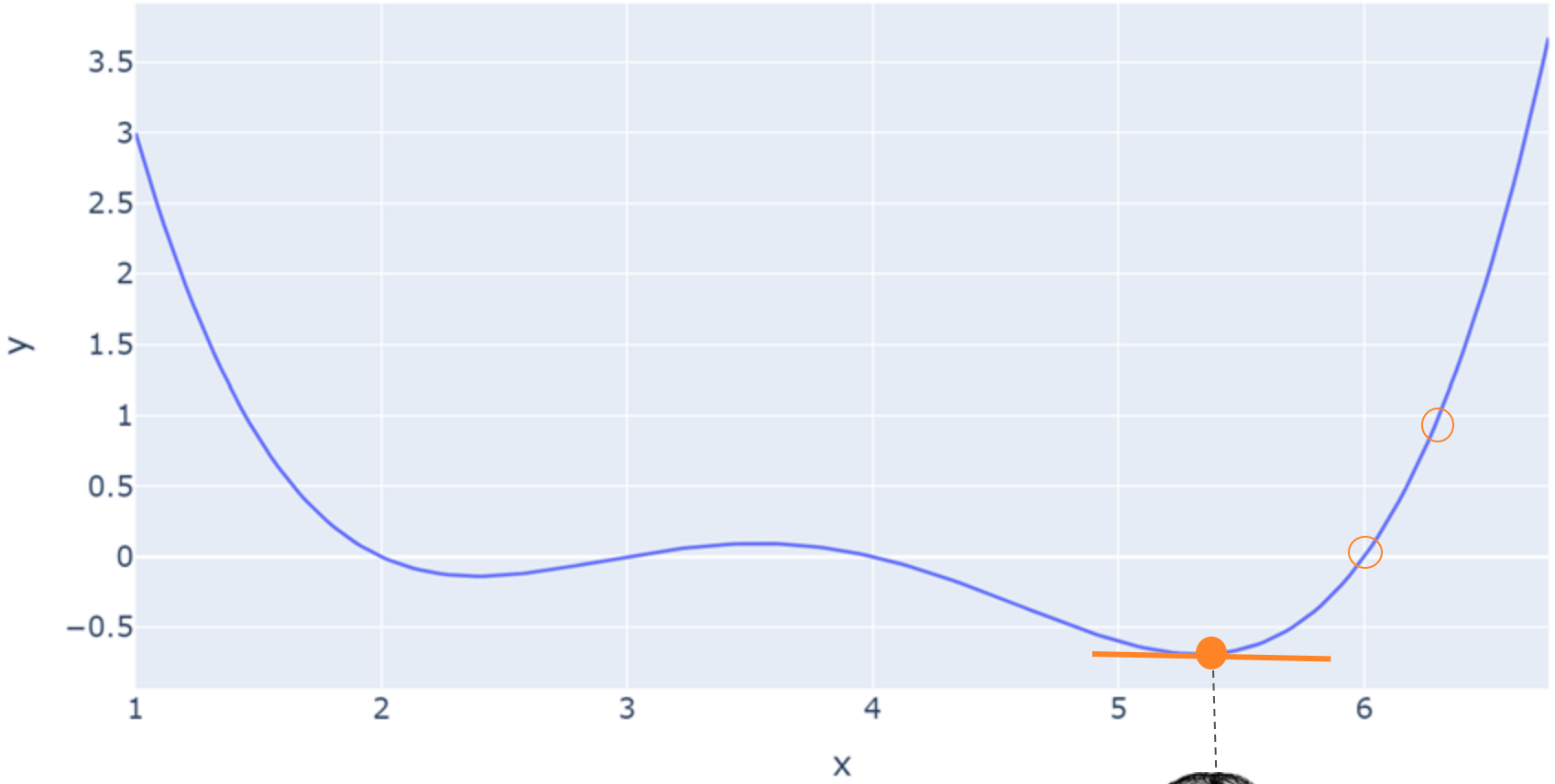
FINDING THE MINIMUM, REDUX

Identify the downhill direction; take another step left or right to move downhill.



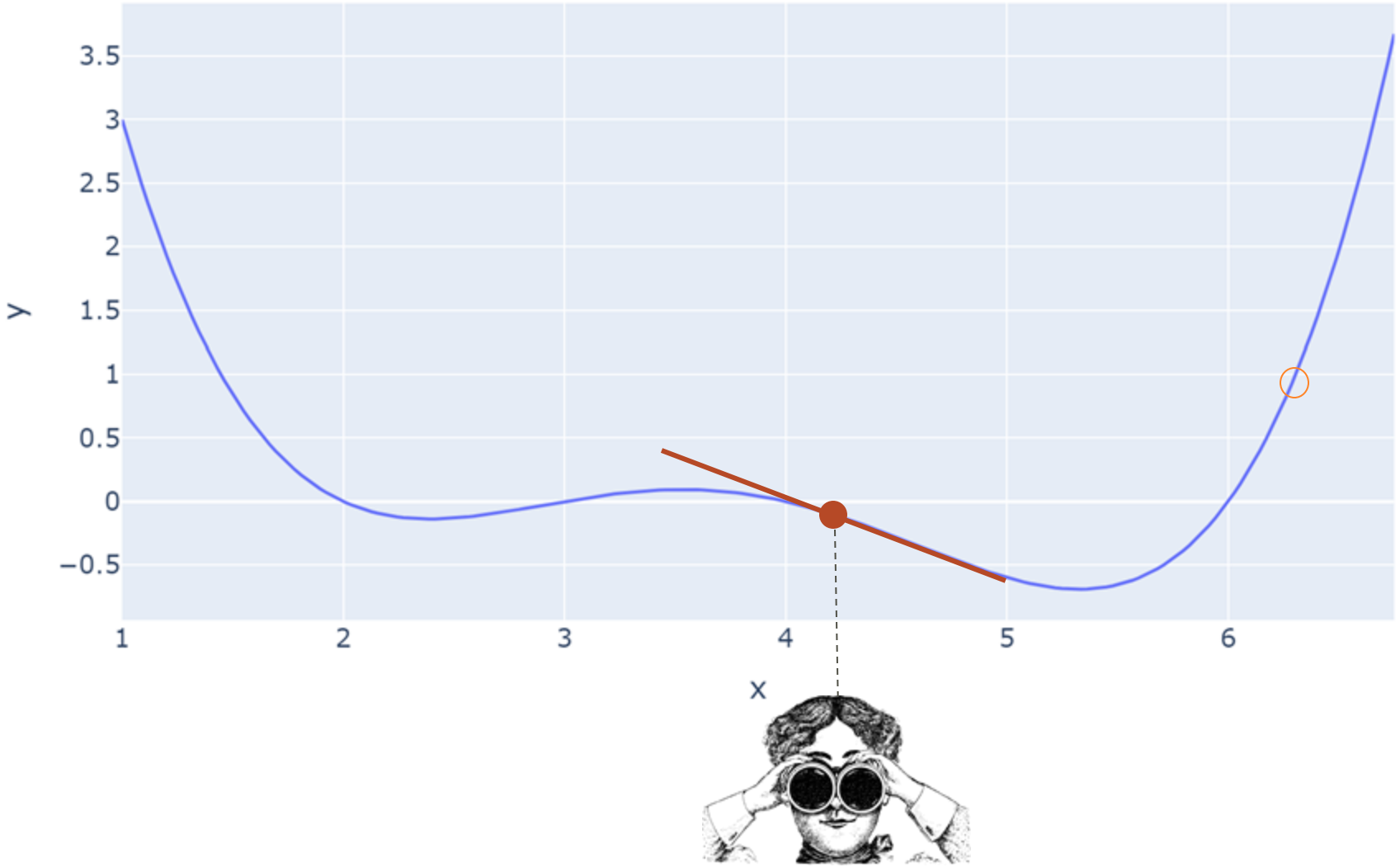
FINDING THE MINIMUM, REDUX

Repeat until the tangent slope is close to 0. (How close? We choose!)



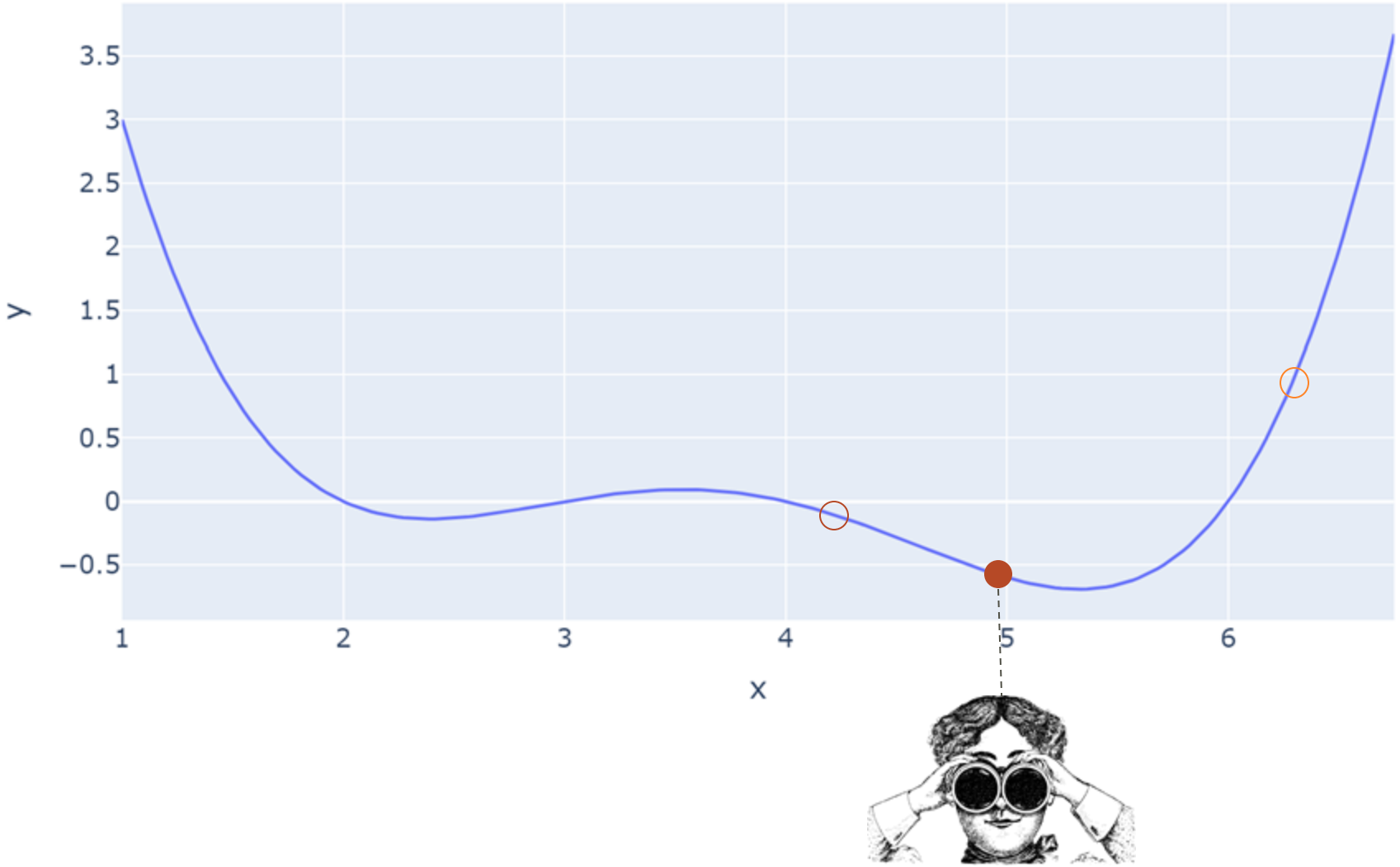
FINDING THE MINIMUM, REDUX

What if we had started elsewhere?



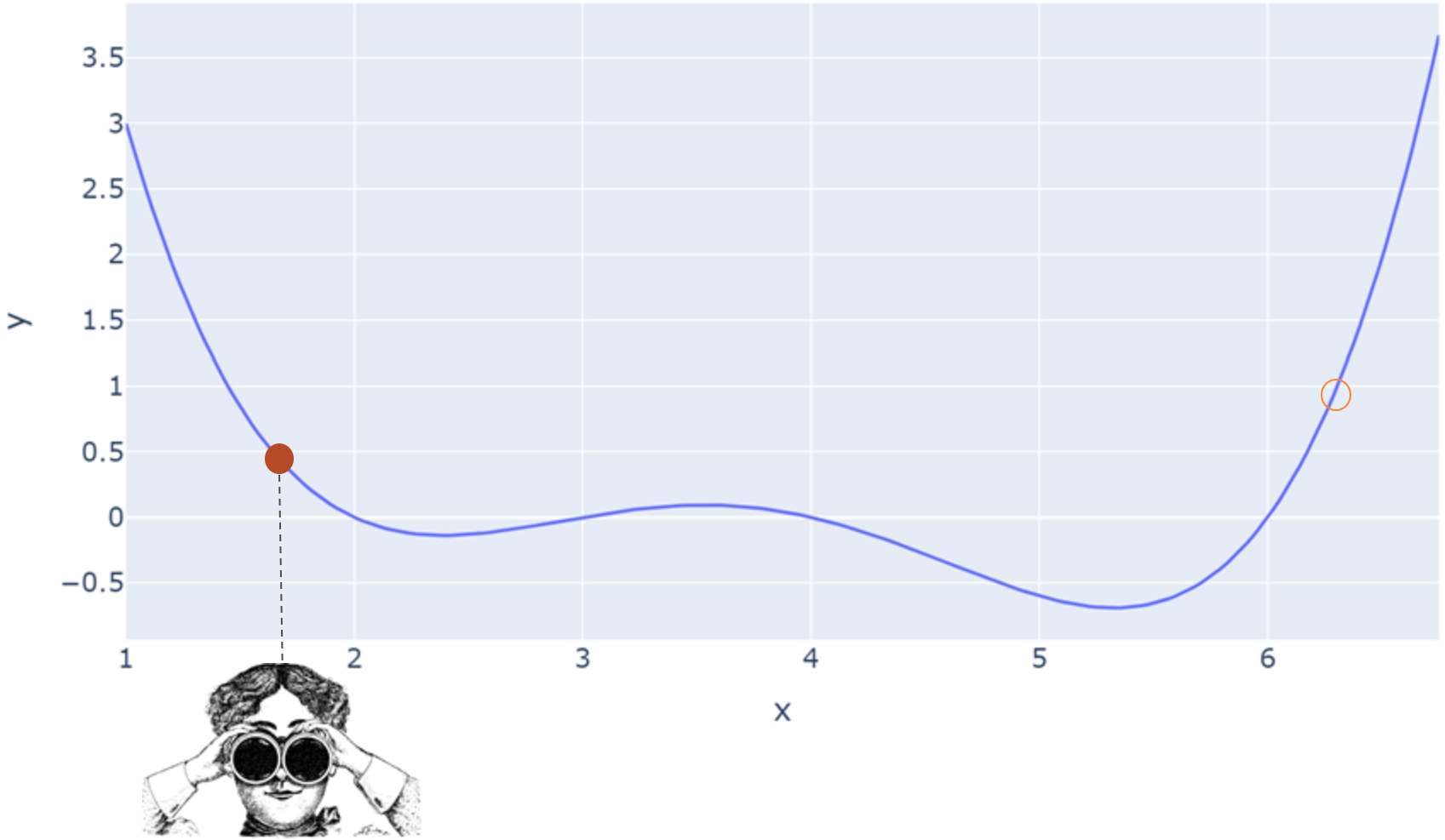
FINDING THE MINIMUM, REDUX

Take a step **right** to move the point downhill, and iterate like before.



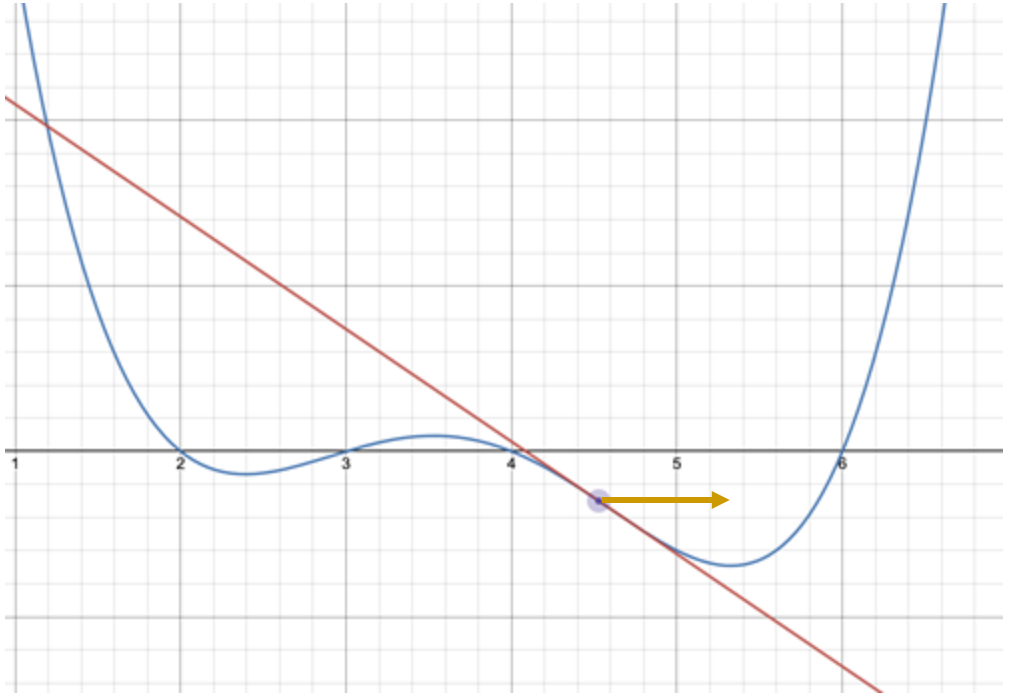
FINDING THE MINIMUM, REDUX

What if we had started over here? We might not end up at the same minimum!

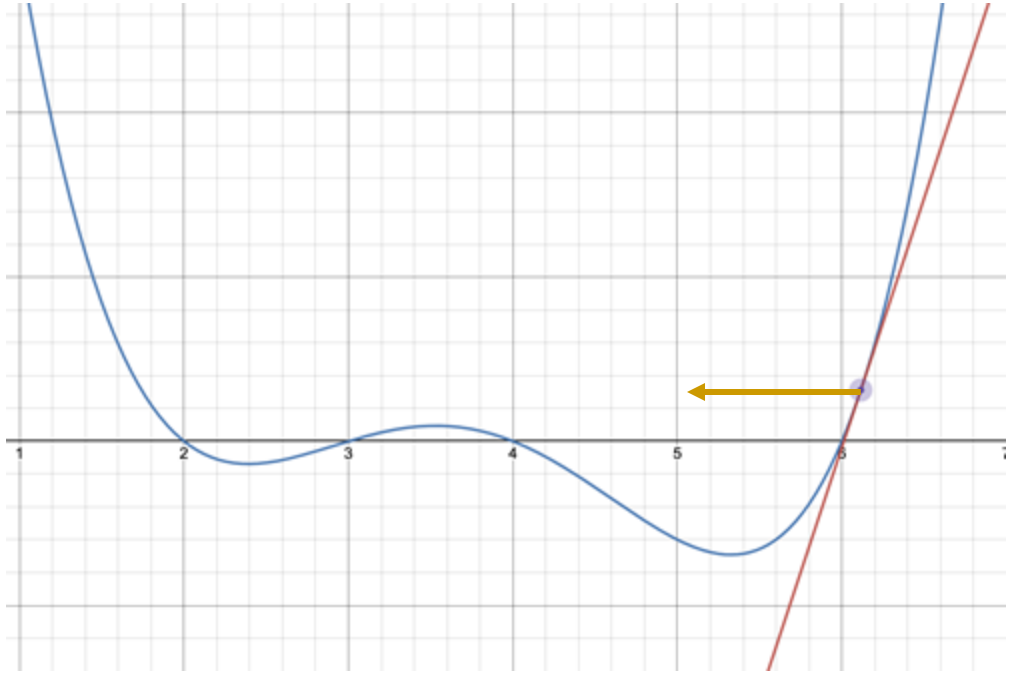


SLOPES TELL US WHERE TO MOVE ALONG THE HORIZONTAL AXIS

Negative slope → step to the right
Move in the *positive* direction



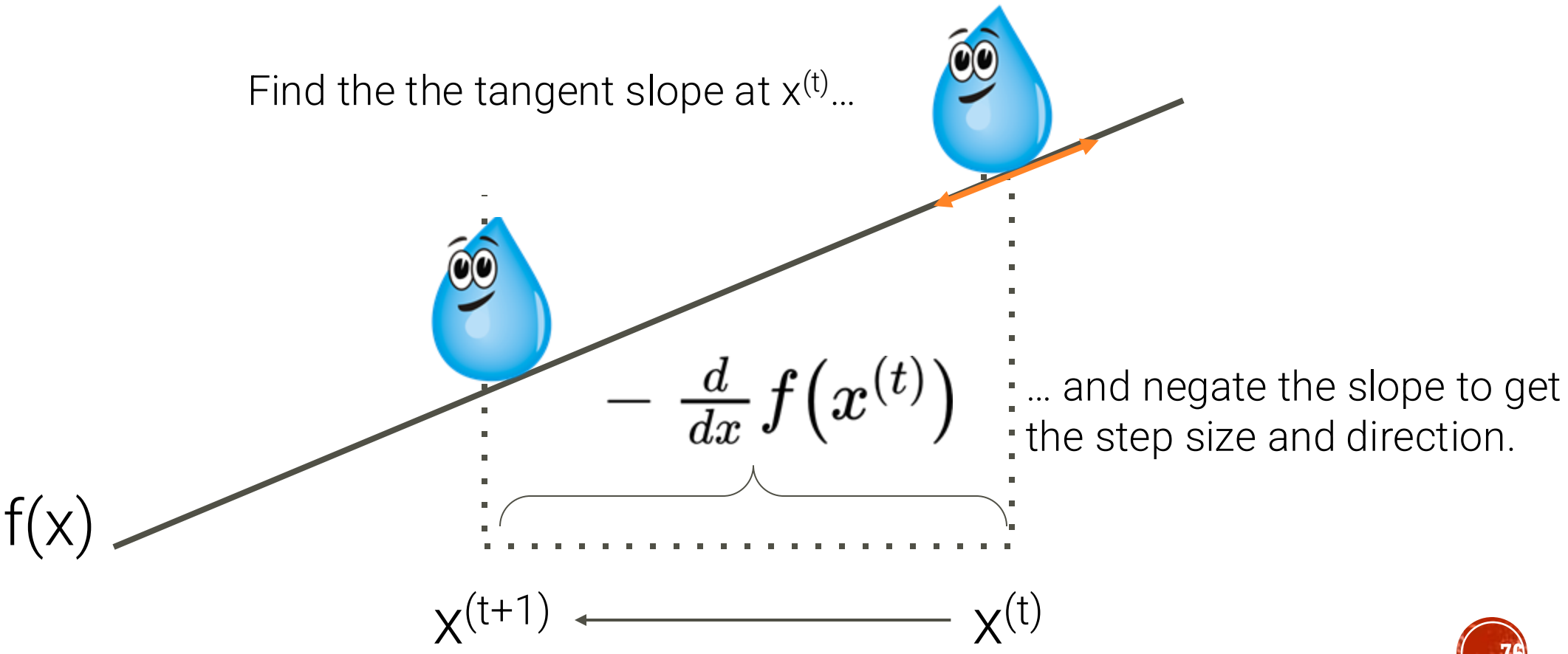
Positive slope → step to the left
Move in the *negative* direction



The derivative of the function at each point tells us the direction of our next guess.
Demo link (adjust the slider!): https://www.desmos.com/calculator/twpnylu4lr

WHAT IF OUR STEP SIZE HAD THE SAME VALUE AS THE SLOPE?

$$x^{(t+1)} = x^{(t)} - \frac{d}{dx} f\left(x^{(t)}\right)$$



SLOPES TELL US WHERE TO GO

The derivative of the function at each point tells us the direction of our next guess.

Negative slope → step to the right
Move x in the *positive* direction

Positive slope → step to the left
Move x in the *negative* direction

A starting algorithm: **Take a step with same size as the slope, but in the opposite direction.**

$$x^{(t+1)} = x^{(t)} - \frac{d}{dx} f\left(x^{(t)}\right)$$

Our next guess for the minimizing x

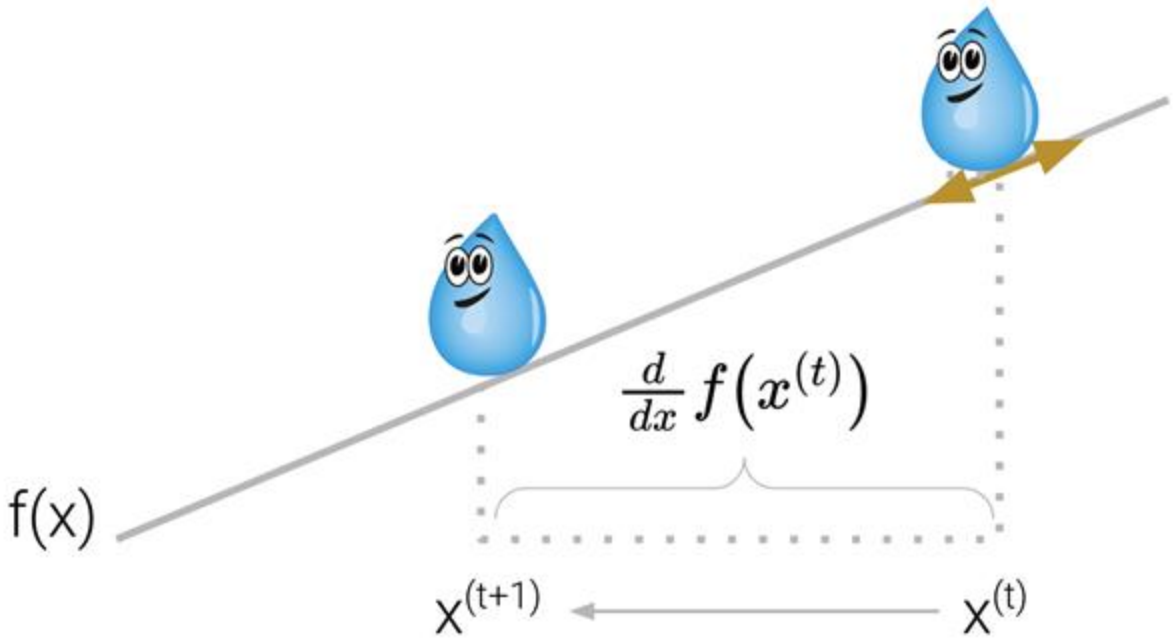
Start at the previous guess

…move opposite to the slope

INITIAL ALGORITHM FOR FINDING A MINIMUM

At each iteration, take a step with same size as the slope, but in the opposite direction.

$$x^{(t+1)} = x^{(t)} - \frac{d}{dx} f\left(x^{(t)}\right)$$



- lec13.ipynb

INTRODUCING A LEARNING RATE

Problem: Each step is too big, so we overshoot the minimizing x.
Solution: *Decrease the size* of each step.

Updated algorithm: α represents a **learning rate** that we choose. It controls the size of each step.

$$x^{(t+1)} = x^{(t)} - \alpha \frac{d}{dx} f\left(x^{(t)}\right)$$

Our next guess for the minimizing x

…starts at the previous guess

…and moves opposite to the slope

For now, the learning rate is constant. α could also decrease over time, or *decay*.

INTRODUCING A LEARNING RATE

When do we **stop updating**?

- After a fixed number of updates
- Subsequent update doesn't change "much"

Convergence: When the algorithm settles on a solution and stops updating significantly (or at all)

$$x^{(t+1)} = x^{(t)} - \alpha \frac{d}{dx} f\left(x^{(t)}\right)$$

AN IMPROVED ALGORITHM FOR FINDING A MINIMUM

At each iteration, take a step **proportional in size to the slope**, but in the opposite direction.

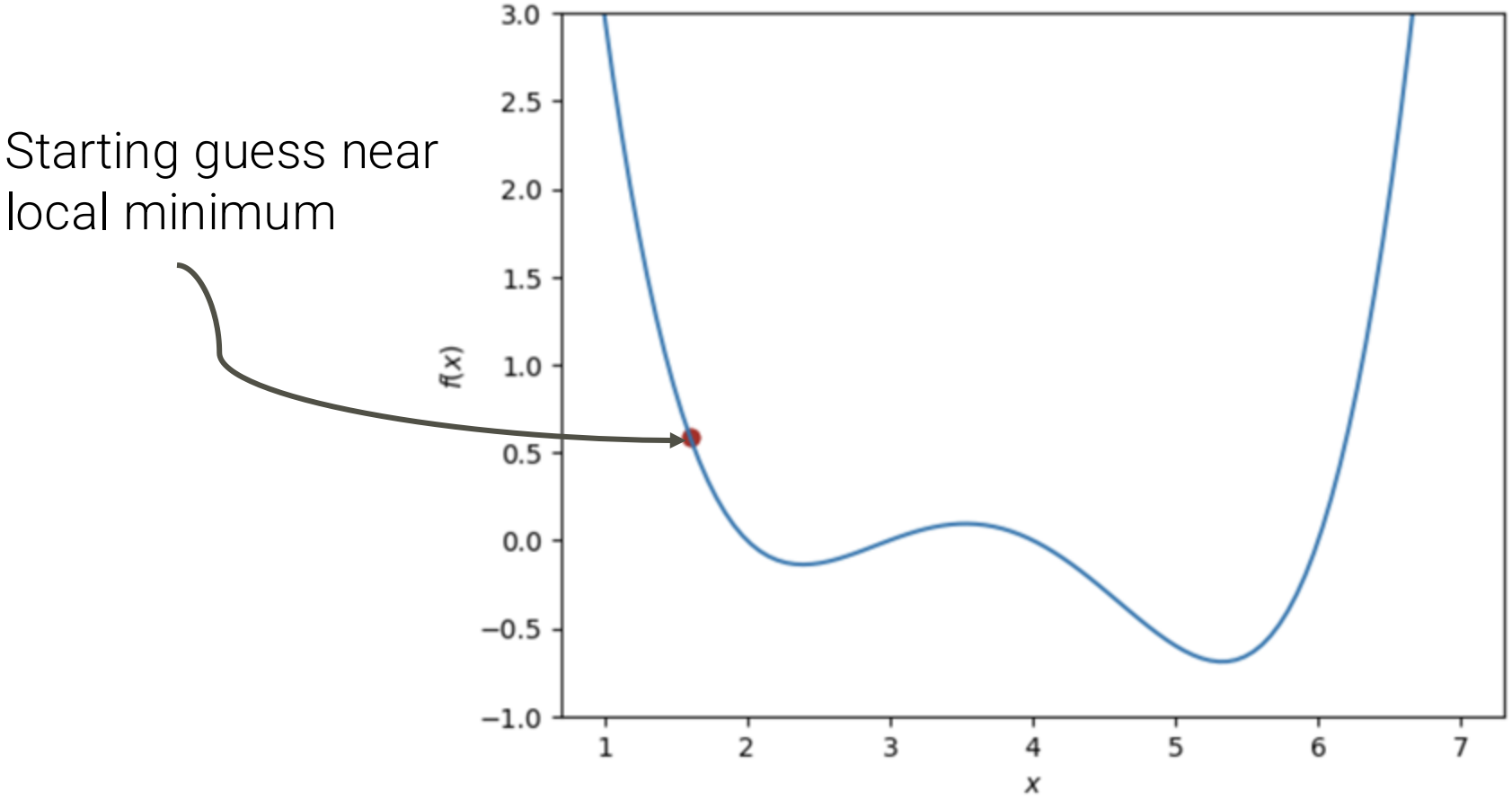
Adjust the step size using the **learning rate** α .

$$x^{(t+1)} = x^{(t)} - \alpha \frac{d}{dx} f\left(x^{(t)}\right)$$

- lec13.ipynb

MULTIPLE MINIMA

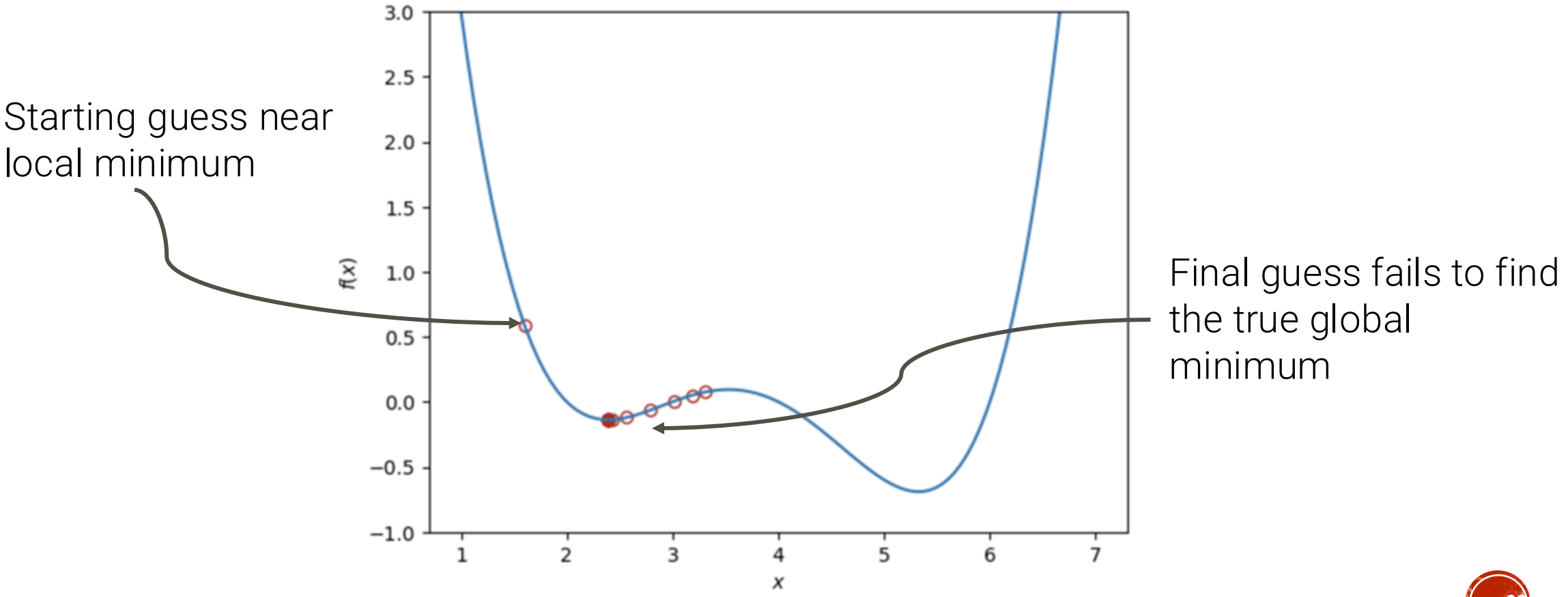
What if our initial guess had been elsewhere?



MULTIPLE MINIMA

What if our initial guess had been elsewhere?

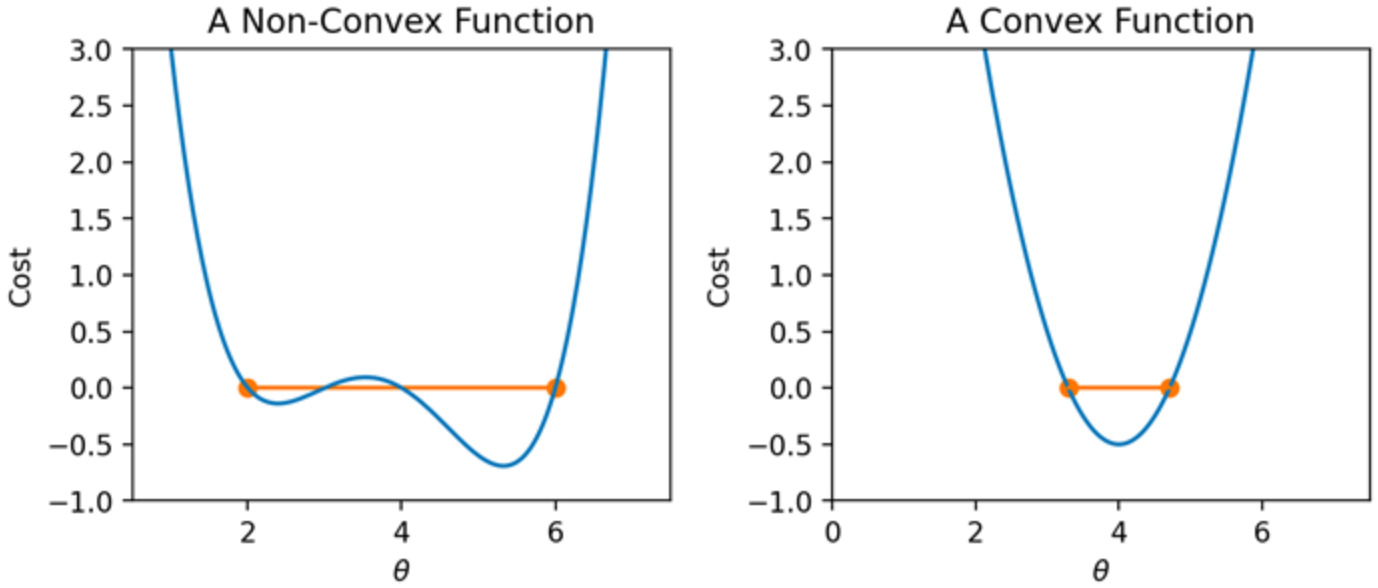
The algorithm may have gotten "stuck" in a local minimum.



CONVEXITY

In plain English: If I draw a line between any two points on a convex curve, all values on the curve must be at or below the line.

$$tf(a) + (1-t)f(b) \geq f(ta + (1-t)b)$$
$$\text{For all } a, b \text{ in domain of } f \text{ and } t \in [0, 1]$$

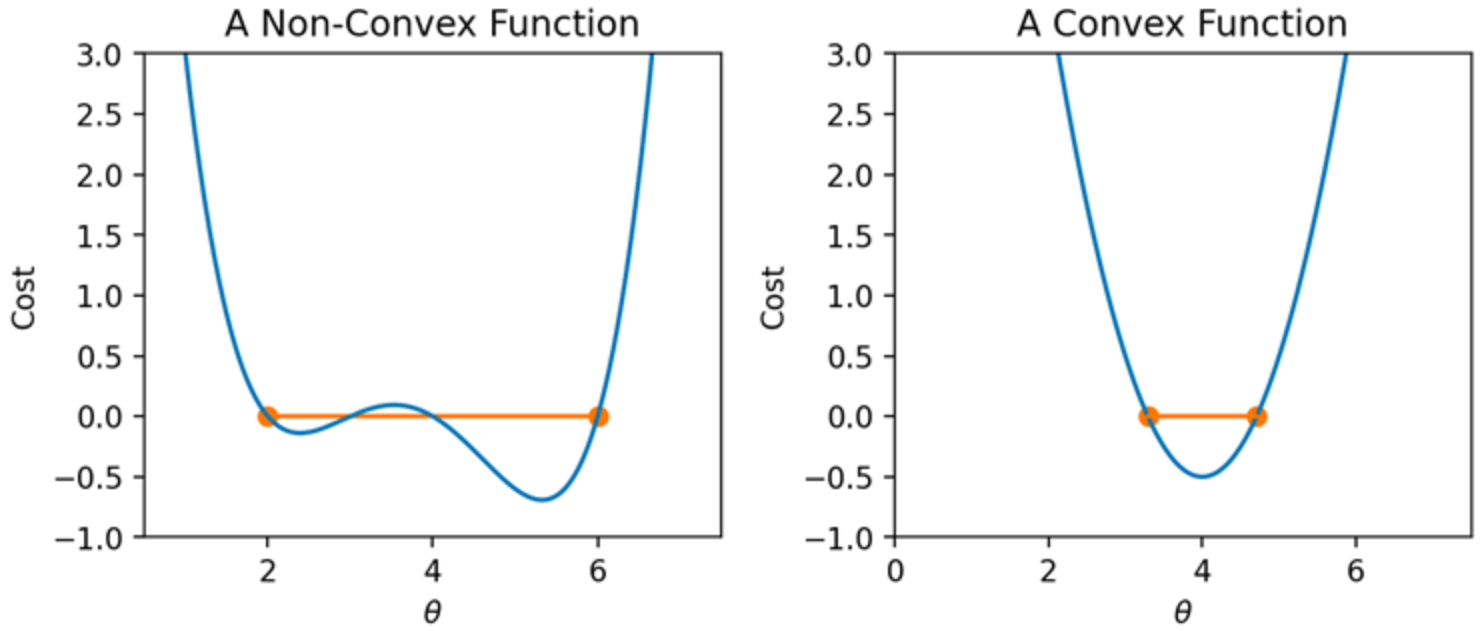


For a **convex** function, **any local minimum is a global minimum.**

- The "arbitrary function" in the earlier demo is non-convex
- **MSE *is* convex**, which is why it is a popular choice of loss function

CONVEXITY

Gradient descent is **only guaranteed to converge for convex functions** (given enough iterations and an appropriate step size).



FROM ARBITRARY FUNCTIONS TO LOSS FUNCTIONS

Recall: When modeling, we aim to identify the model parameters that minimize a *loss function*.

Terminology clarification:

In past lectures, **"loss"** referred to the error on a **single data point,** e.g., $L(y, \hat{y}) = (y - \hat{y})^2$

In applications, we usually care more about the **average error across all data points**

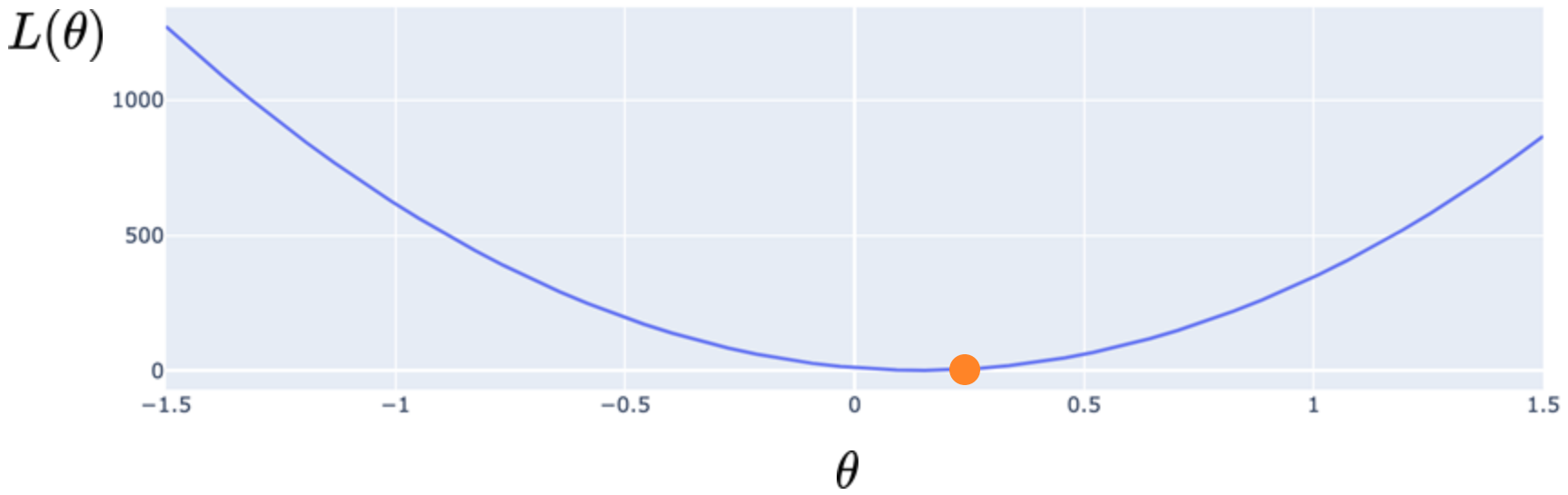
Moving forward, **we will most often use "loss" to refer to a model's average error across the dataset.** Also known as the empirical risk (R), cost function, or objective function.

$$L(\theta) = R(\theta) = \frac{1}{n} \sum_{i=1}^{n} l(y, \hat{y})$$

OPTIMIZATION ON LOSS FUNCTIONS

Recall: When modeling, we aim to identify the model parameters that minimize a *loss function*.

Goal: choose the value of θ that minimizes $L(\theta)$, the model's **loss** on the dataset



FROM ARBITRARY FUNCTIONS TO LOSS FUNCTIONS

Goal: Choose the value of θ that minimizes $L(\theta)$, the model's loss on the dataset

Gradient: The direction of steepest ascent for a function at some specific input.

The **1D gradient descent** algorithm:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \frac{d}{d\theta} L\left(\theta^{(t)}\right)$$

Earlier algorithm, for reference:

$$x^{(t+1)} = x^{(t)} - \alpha \frac{d}{dx} f\left(x^{(t)}\right)$$

GRADIENT DESCENT ON THE TIPS DATASET

We want to predict the tip (y) given the price of a meal (x).

1. Choose a model: $\hat{y} = \theta_1 x$

2. Choose a loss function: $L(\theta) = MSE(\theta) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \theta_1 x_i)^2$

3. Optimize the model parameter with ~~calculus~~ ~~geometry~~ gradient descent!

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \frac{d}{d\theta} L\left(\theta^{(t)}\right)$$

GRADIENT DESCENT ON THE TIPS DATASET

Our model:

$$\hat{y} = \theta_1 x$$

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \frac{d}{d\theta} L\left(\theta^{(t)}\right)$$

Our loss function:

$$L(\theta) = MSE(\theta) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \theta_1 x_i)^2$$

Take the derivative with respect to θ_1 :

$$\frac{d}{d\theta_1} L\left(\theta_1^{(t)}\right) = \frac{-2}{n}\sum_{i=1}^{n}\left(y_i - \theta_1^{(t)} x_i\right) x_i$$

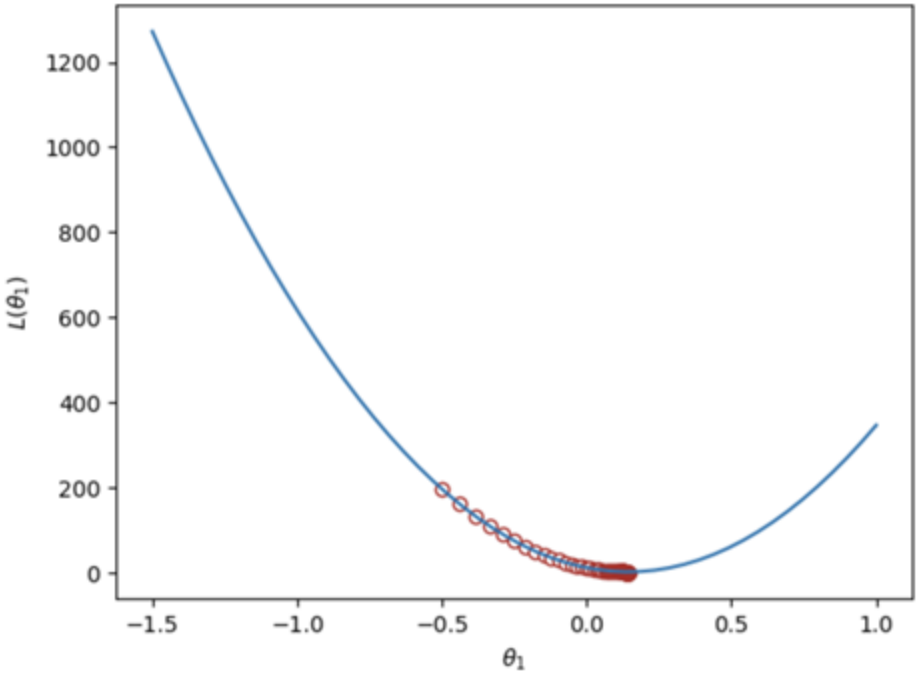
The gradient descent update rule:

$$\theta_1^{(t+1)} = \theta_1^{(t)} - \alpha \frac{-2}{n}\sum_{i=1}^{n}\left(y_i - \theta_1^{(t)} x_i\right) x_i$$

GRADIENT DESCENT (GD) WITH TIPS

Loss function: $MSE(\theta) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \theta_1 x_i)^2$

GD update rule: $\theta_1^{(t+1)} = \theta_1^{(t)} - \alpha\frac{-2}{n}\sum_{i=1}^{n}\left(y_i - \theta_1^{(t)}x_i\right)x_i$



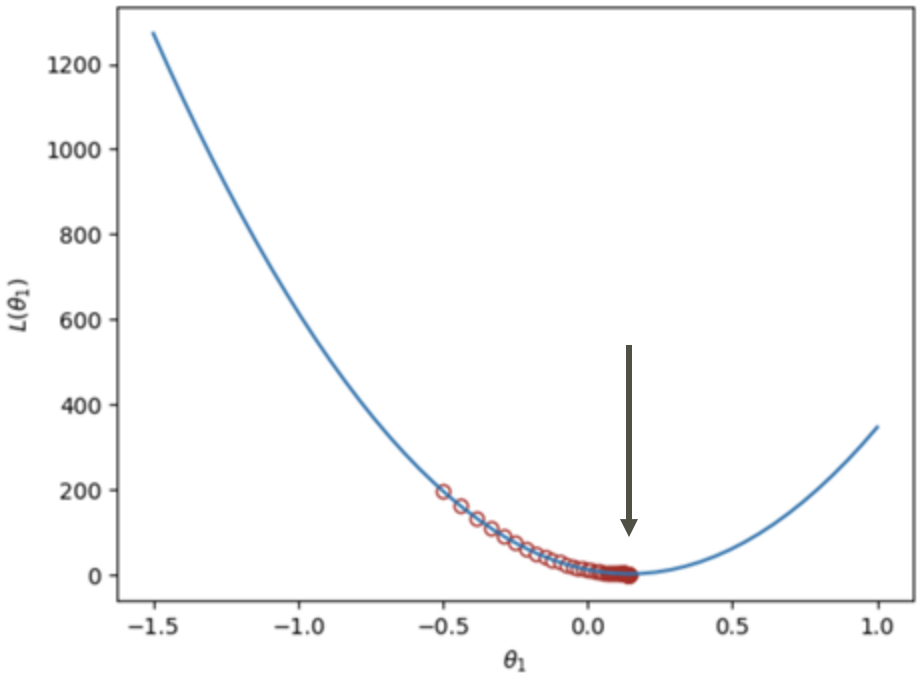
- lec13.ipynb

MSE is minimized when we set $\theta_1 = 0.1437$

MSE IS CONVEX!

When we visualized the MSE loss on the `tips` data, there was a single global minimum.

You will show in HW6 that L2 loss is convex, so gradient descent converges to the minimum.



This is one reason why the MSE is a popular choice of loss function: it behaves "nicely" for optimization

3D GRADIENT DESCENT AT MOUNT DIABLO

Models with more than one parameter

Usually, models will have more than one parameter that needs to be optimized.

Simple linear regression: $\hat{y} = \theta_0 + \theta_1 x$

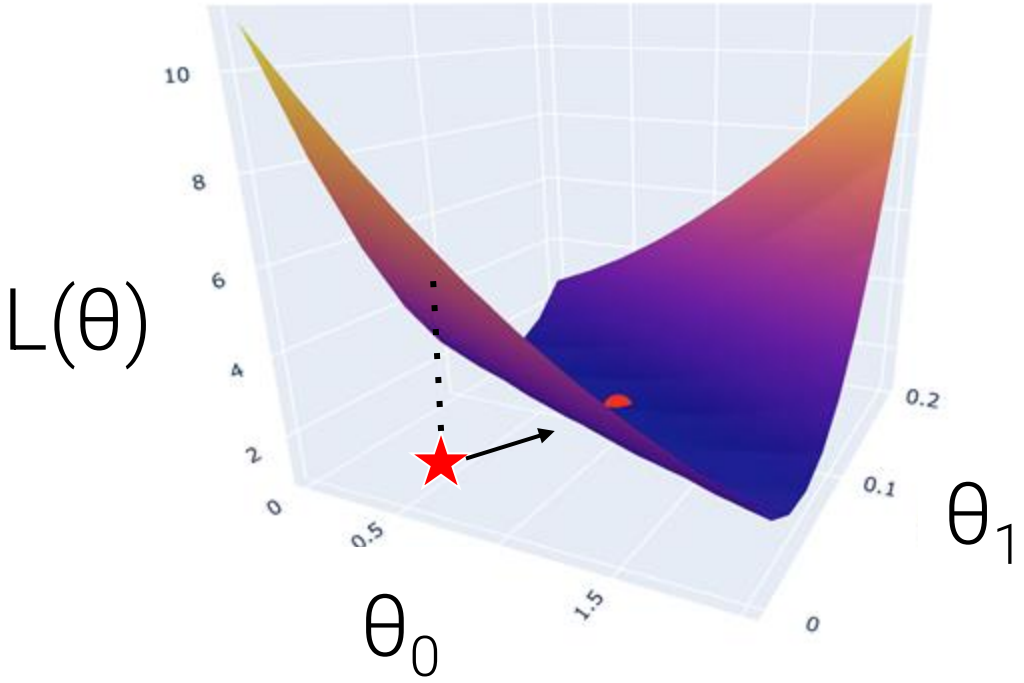
Multiple linear regression: $\hat{\mathbb{Y}} = \theta_0 + \theta_1 \mathbb{X}_{:,1} + \theta_2 \mathbb{X}_{:,2} \ldots + \theta_p \mathbb{X}_{:,p}$

Idea: Expand gradient descent to update our guesses for *all* model parameters, all in one go!

THE GRADIENT VECTOR

Before, the derivative of the loss function guided us towards the minimizing parameter value.

On a 3D (or higher) loss surface, the gradient is described by a vector of partial derivatives.



$-\nabla_{\vec{\theta}} L$ always points towards the **downhill direction** of the surface.

For the *vector* of parameter values:

$$\vec{\theta} = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix}$$

Find the *partial derivative* of loss with respect to each parameter:

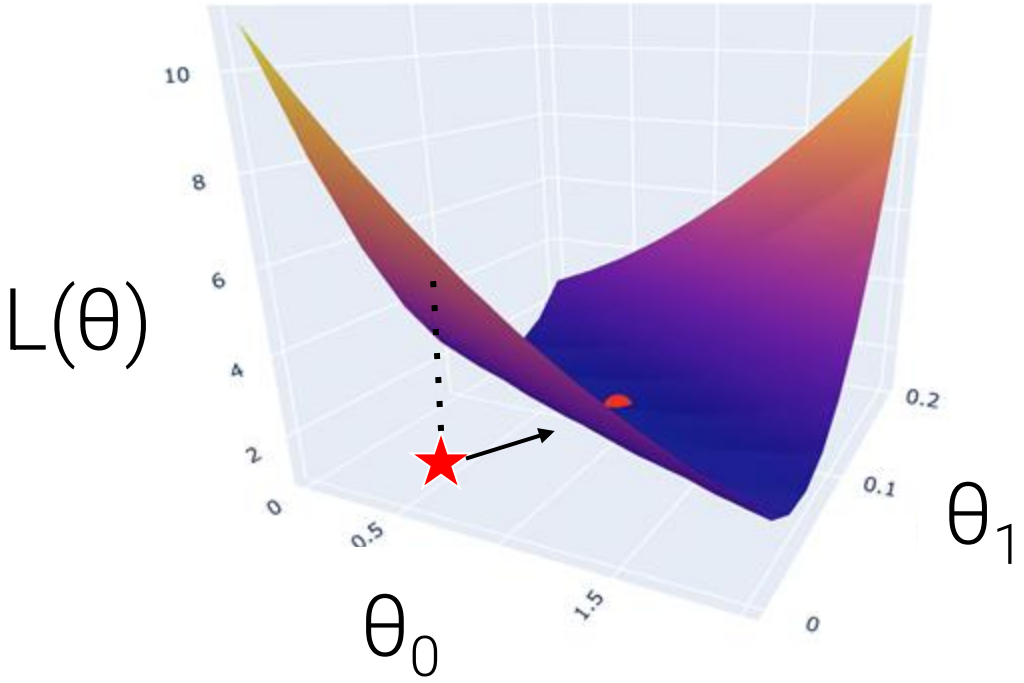
$$\nabla_{\vec{\theta}} L(\theta_0, \theta_1) = \begin{bmatrix} \frac{\partial}{\partial \theta_0} L(\theta_0, \theta_1) \\ \frac{\partial}{\partial \theta_1} L(\theta_0, \theta_1) \end{bmatrix}$$

Note: The 2D gradient is not tangent to the 3D loss surface!

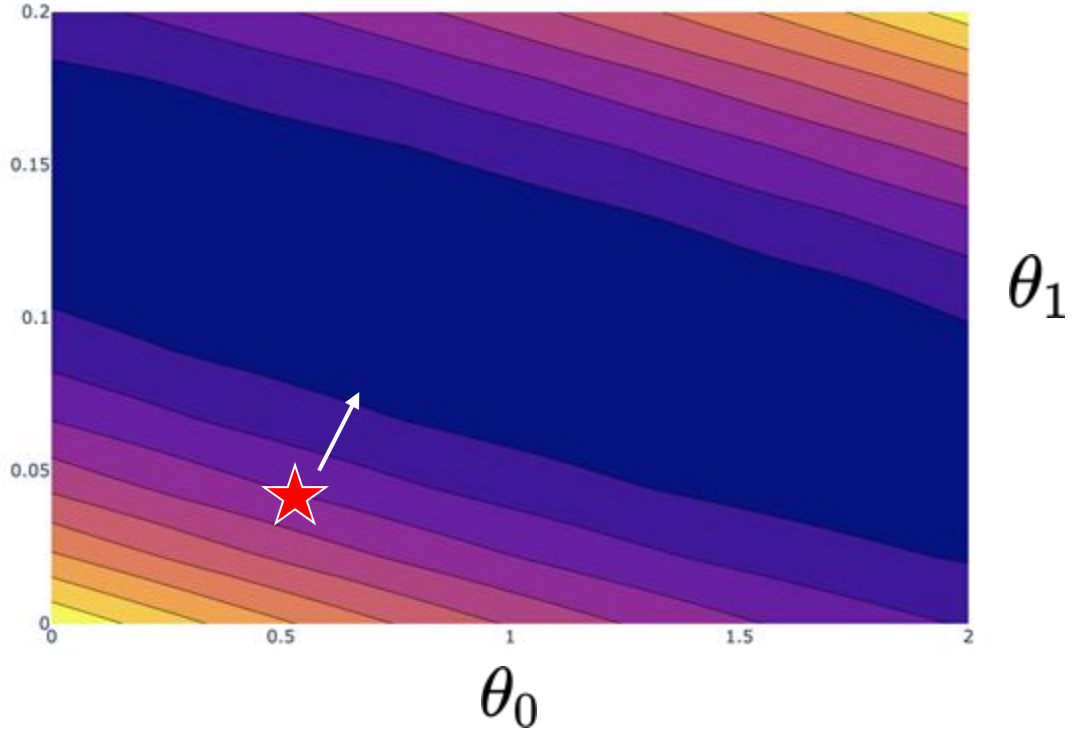
THE GRADIENT VECTOR

Before, the derivative of the loss function guided us towards the minimizing parameter value.

On a 3D (or higher) loss surface, the **gradient** is described by a **vector** of partial derivatives.



A bird's eye view from above:



$-\nabla_{\vec{\theta}} L$ always points towards the **downhill direction** of the surface.

Note: The 2D gradient is not tangent to the 3D loss surface!

EXAMPLE: GRADIENT OF A 2D FUNCTION

Consider the 2D function: $f(\theta_0, \theta_1) = 8\theta_0^2 + 3\theta_0\theta_1$

For a function of 2 variables $f(\theta_0, \theta_1)$ we define the gradient:

$$\frac{\partial f}{\partial \theta_0} = 16\theta_0 + 3\theta_1$$

$$\frac{\partial f}{\partial \theta_1} = 3\theta_0$$

$$\nabla_{\vec{\theta}} f(\vec{\theta}) = \begin{bmatrix} 16\theta_0 + 3\theta_1 \\ 3\theta_0 \end{bmatrix}$$

EXAMPLE: GRADIENT OF A FUNCTION IN COLUMN VECTOR NOTATION

For a generic function of p + 1 variables:

$$\nabla_{\vec{\theta}} f(\vec{\theta}) = \begin{bmatrix} \frac{\partial}{\partial \theta_0}(f) \\ \frac{\partial}{\partial \theta_1}(f) \\ \vdots \\ \frac{\partial}{\partial \theta_p}(f) \end{bmatrix}$$

HOW TO INTERPRET GRADIENTS

You should read these gradients as:

- If I were to barely increase the 1st parameter's value, what happens to the output?
- If I were to barely increase the 2nd parameter's value, what happens to the output?
- And so on.

$$\nabla_{\vec{\theta}} f(\vec{\theta}) = \begin{bmatrix} \frac{\partial}{\partial \theta_0}(f) \\ \frac{\partial}{\partial \theta_1}(f) \\ \vdots \\ \frac{\partial}{\partial \theta_p}(f) \end{bmatrix}$$

GRADIENT DESCENT IN MULTIPLE DIMENSIONS

Recall our 1D update rule:

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \frac{d}{d\theta} L\left(\theta^{(t)}\right)$$

Now, for models with multiple parameters, we work in terms of vectors:

$$\begin{bmatrix} \theta_0^{(t+1)} \\ \theta_1^{(t+1)} \\ \vdots \end{bmatrix} = \begin{bmatrix} \theta_0^{(t)} \\ \theta_1^{(t)} \\ \vdots \end{bmatrix} - \alpha \begin{bmatrix} \frac{\partial L}{\partial \theta_0} \\ \frac{\partial L}{\partial \theta_1} \\ \vdots \end{bmatrix}$$

Written in a more compact form:

$$\vec{\theta}^{(t+1)} = \vec{\theta}^{(t)} - \alpha \nabla_{\vec{\theta}} L\left(\vec{\theta}^{(t)}\right)$$

READING MATERIAL

- Chapter# 4: Principle of Data Science
- Chapter# 2 & 4: Hands-On ML [Aurélien]